

[\[Table of Contents\]](#) | [\[UAAG 2.0 Reference\]](#)



User Agent Accessibility Guidelines (UAAG) 2.0

W3C Working Group Note 15 December 2015

This version:

<http://www.w3.org/TR/2015/NOTE-UAAG20-20151215/>

Latest version:

<http://www.w3.org/TR/UAAG20/>

Previous version:

<http://www.w3.org/TR/2015/WD-UAAG20-20150915/>

Latest Editor's Draft:

<http://www.w3.org/TR/2015/NOTE-UAAG20-20151215/>

Editors:

James Allan, Texas School for the Blind and Visually Impaired

Greg Lowney, Lowney Access Research

Kim Patch, Redstart Systems

Jeanne Spellman, W3C/Web Accessibility Initiative

Copyright © 2015 [W3C](#)® ([MIT](#), [ERCIM](#), [Keio](#), [Beihang](#)). W3C [liability](#), [trademark](#) and [document use](#) rules apply.

Abstract

UAAG 2.0 guides developers in designing user agents that make the web more accessible to people with disabilities. User agents include browsers, browser extensions, media players, readers and other applications that render web content. A user agent that follows UAAG 2.0 will improve accessibility through its own user interface and its ability to communicate with other technologies, including assistive technologies. UAAG and supporting resources are also intended to meet the needs of different audiences, including developers, policy makers, and managers. All users, not just users with disabilities, will benefit from user agents that follow UAAG 2.0.

In addition to helping developers of browsers and media players, UAAG 2.0 benefits developers of assistive technologies because it explains what types of information and control an assistive technology can expect from a user agent that follows UAAG 2.0. Assistive technologies not addressed directly by UAAG 2.0 (e.g. braille rendering) are still essential to ensuring web access for some users with disabilities.

The "User Agent Accessibility Guidelines 2.0" (UAAG 2.0) is part of a series of accessibility guidelines published by the [W3C Web Accessibility Initiative \(WAI\)](#). UAAG is introduced in the [User Agent Accessibility Guidelines \(UAAG\) Overview](#).

Status of this Document

May be Superseded

This section describes the status of this document at the time of its publication. Other documents may supersede this document. A list of current [W3C publications](#) and the latest revision of this technical report can be found in the [W3C technical reports index](#) at <http://www.w3.org/TR/>.

W3C Working Group Note of UAAG 2.0

This is a W3C Working Group Note of 15 December 2015 from the [User Agent Accessibility Guidelines Working Group](#) (UAWG). This Note was Recommendation track work that closed before entering the Candidate Recommendation (CR) phase. Contributing factors included insufficient resources for formal CR testing. A partial test suite is available, and at least two candidate implementations have been identified for 90% of the features, but not formally tested.

- UAAG 2.0 completed [Last Call](#) in 2013 and UAWG has published 2 subsequent working drafts in response to comments.
- UAAG 2.0 has received [wide review](#) with 246 issues raised from 37 organizations or individuals representing browsers, media players, W3C working groups, and accessibility experts.
- All comments and issues have been resolved and there are no outstanding objections to UAAG 2.0.
- There are 3 features at risk where no implementations have been identified
 - 1.4.5 Default to Platform Text Settings (examples of high contrast, but not text settings)
 - 2.2.4 Options for Wrapping in Navigation
 - 3.1.6 Form Submission Confirm

UAAG 2.0 addresses barriers for people with disabilities that could be improved or solved with greater user agent accessibility. UAWG hopes that publication of this Note will provide user agent developers, people with disabilities, accessibility experts, and other involved parties a useful reference for improving the accessibility of user agents.

Important changes to this document since the previous version are highlighted below. A [diff document of all changes](#) is available.

- Changed term "Content Specifications" in 5.1.2 Implement Accessibility Features of Content Specifications to "Web Content Technology Specifications"

Comments on the Note should be sent to public-uaag2-comments@w3.org ([Public Archive](#)). Although the UAWG is closing and will not respond to comments, comments can provide useful input for future work in this area.

Web Accessibility Initiative

This document has been produced as part of the [W3C Web Accessibility Initiative](#) (WAI). The goals of the User Agent Working Group (UAWG) are discussed in the [Working Group charter](#).

No Endorsement

Publication as a Working Group Note does not imply endorsement by the W3C Membership. This is a draft document and may be updated, replaced or obsoleted by other documents at any time. It is inappropriate to cite this document as other than work in progress.

Patents

This document was produced by a group operating under the [5 February 2004 W3C Patent Policy](#). W3C maintains a [public list of any patent disclosures](#) made in connection with the deliverables of the group; that page also includes instructions for disclosing a patent. An individual who has actual knowledge of a patent which the individual believes contains [Essential Claim\(s\)](#) must disclose the information in accordance with [section 6 of the W3C Patent Policy](#).

This document is governed by the [1 September 2015 W3C Process Document](#).

Table of Contents

Abstract

Status of this Document

Introduction

Overview

UAAG 2.0 Layers of Guidance

UAAG 2.0 Supporting Documents

Components of Web Accessibility

Levels of Conformance

Relationship with Web Content Accessibility Guidelines(WCAG) 2.0

Relationship with Authoring Tool Accessibility Guidelines (ATAG) 2.0

UAAG 2.0 Guidelines

UAAG 2.0 Conformance Applicability Notes

PRINCIPLE 1: Perceivable

Guideline 1.1: Alternative content

Guideline 1.2: Missing content

Guideline 1.3: Highlighting

Guideline 1.4: Text configuration

Guideline 1.5: Volume configuration

Guideline 1.6: Synthesized speech configuration

Guideline 1.7: User style sheet configuration

Guideline 1.8: Orientation in viewports

Guideline 1.9: Alternative views

Guideline 1.10: Element Information

PRINCIPLE 2: Operable

Guideline 2.1: Keyboard access

Guideline 2.2: Sequential navigation

Guideline 2.3: Direct navigation and activation

Guideline 2.4: Text Search

Guideline 2.5: Structural navigation

Guideline 2.6: Preference settings

Guideline 2.7: Graphical controls

Guideline 2.8: Time-independent interaction

Guideline 2.9: Flashing

Guideline 2.10: Time-based media

Guideline 2.11: Other Input Devices

PRINCIPLE 3: Understandable

Guideline 3.1: Mistakes

Guideline 3.2: Documentation

Guideline 3.3: Predictable

PRINCIPLE 4: Programmatic access

Guideline 4.1: Assistive technology

PRINCIPLE 5: Specifications and conventions

Guideline 5.1: Follow specifications

Conformance

Appendix A: Glossary

Appendix B: How to refer to UAAG 2.0 from other documents

Appendix C: References

Appendix D: Acknowledgements

Introduction

This document is divided into two types of sections. The normative sections are: principles, guidelines, success criteria, notes, conformance, and glossary. These are required to claim conformance to UAAG 2.0. The other sections of this document, including this introduction, Appendix B, C and D, are informative. They explain and amplify the normative sections.

A user agent is any software that retrieves, renders and facilitates end-user interaction with web content. User agents include web browsers, media players, add-ons (plug-ins & extensions), and web applications that help in retrieving, rendering and interacting with web content. UAAG 2.0 specifies requirements for user agent developers that will lower barriers to accessibility.

For an introduction to UAAG, see the [User Agent Accessibility Guidelines \(UAAG\) Overview](#).

Overview

Improving accessibility means considering a wide range of disabilities. These include visual, auditory, physical, speech, cognitive, language, learning, neurological disabilities, and disabilities related to aging. The goal of UAAG 2.0 is to ensure that all users, including users with disabilities, have equal control over the environment they use to access the web.

Some users have more than one disability, and the needs of different disabilities can conflict. Many UAAG 2.0 requirements use configuration preferences to ensure that a feature designed to improve accessibility for one user does not interfere with the needs of another user. To avoid overwhelming users with an abundance of configuration options, UAAG 2.0 includes requirements that promote clear documentation and ease of configuration.

The UAWG expects that software that satisfies the requirements of UAAG 2.0 will be more flexible, manageable, extensible, and beneficial for a broad range of users.

UAAG 2.0 Layers of Guidance

In order to meet the needs of different audiences, UAAG provides three layers of guidance: overall **principles**, general **guidelines**, and testable **success criteria**. There is more detail for each success criterion in a separate document, [UAAG 2.0 Reference](#), including explanatory intent, examples of how the criterion can apply in different user situations, and links to resources.

1. **Principles** – Five principles provide a foundation for accessible user agents. Principles 1, 2, and 3 are parallel to [Web Content Accessibility Guidelines \(WCAG\) 2.0](#). Principles 4 and 5 are specific to user agents.
 - Principle 1 ensures that the user agent is perceivable, so users can access user agent output

- Principle 2 ensures that the user agent is operable, so users can communicate with the user agent
 - Principle 3 ensures that the user agent is understandable, so users know what to do to use the user agent
 - Principle 4 ensures that assistive technologies can access user agent controls
 - Principle 5 ensures that user agents comply with other accessibility specifications (e.g WCAG) and platform conventions (e.g. Windows, iOS, Linux, Blackberry).
2. **Guidelines** – Under each principle is a set of guidelines for making user agents more accessible to users with disabilities. These guidelines provide a framework to help authors understand the objectives for success criteria so they can better implement them.
 3. **Success Criteria** – Under each guideline is a set of testable success criteria that can be used wherever conformance testing is necessary, including design specification, purchasing, regulation, and contractual agreements.

Each success criterion is assigned a level. The levels are designed to meet the needs of different groups and different situations: A (minimum conformance), AA (recommended conformance), and AAA (advanced conformance). Additional information on UAAG levels can be found in the [Levels of Conformance](#) section.

UAAG 2.0 Supporting Documents

A separate document, entitled **UAAG 2.0 Reference: Explanations, Examples, and Resources for User Agent Accessibility Guidelines 2.0** (hereafter referred to as the "Reference document") provides explanations and examples of how each success criteria might be satisfied. It also includes references to other accessibility resources (such as platform-specific software accessibility guidelines) that provide additional information on how a user agent can satisfy each success criteria. The examples in *UAAG 2.0 Reference* are informative only. Other strategies can be used or required to satisfy the success criteria. The UAAG expects to update *UAAG 2.0 Reference* more frequently than the *User Agent Accessibility Guidelines 2.0*. Developers, W3C Working Groups, users, and others are encouraged to contribute examples and resources to *UAAG 2.0 Reference*.

Components of Web Accessibility

Web accessibility depends on both accessible user agents and accessible content. The level of accessibility of content is largely influenced by the authoring tool used to create it. For an overview of how these components of web development and interaction work together, see

- [Essential Components of Web Accessibility](#)
- [Web Content Accessibility Guidelines \(WCAG\) Overview](#)
- [Authoring Tool Accessibility Guidelines \(ATAG\) Overview](#)

Additional information about the relationship between UAAG 2.0 and ATAG 2.0, and the relationship between UAAG 2.0 and WCAG 2.0, is in the sections on [Relationship with ATAG](#) and [Relationship with WCAG](#).

Levels of Conformance

User agents can conform to UAAG 2.0 at one of three conformance levels: levels A (minimum), AA (recommended), and AAA (advanced). The three levels of UAAG 2.0 conformance are based on the corresponding level designations (A, AA, or AAA) of the individual success criteria (i.e., specific requirements). The user agent can conform to a level by meeting the success criteria of that level and the levels below it.

- Level A conformance: The user agent complies with all applicable level A success criteria.
- Level AA conformance: The user agent complies with all applicable level A and AA success criteria
- Level AAA conformance: The user agent complies with all applicable level A, AA, and AAA success criteria.

UAAG 2.0 has many options that can be managed through preference settings.

For details about what each level represents, how the levels were determined, and how user agent developers and managers can use the levels for prioritizing accessibility improvements and designing user interfaces, see [UAAG 2.0 Reference Levels of Conformance](#).

Relationship to the Web Content Accessibility Guidelines (WCAG) 2.0

The W3C recommendation, [Web Content Accessibility Guidelines \(WCAG\)](#), applies to all web content; UAAG provides additional advice on the application user interface.

Some user agents are used to package web content into native applications, especially on mobile platforms. If the finished application is used to retrieve, render, and facilitate end-user interaction with web content of the end-users choosing, then the application should be considered a stand-alone user agent. If the finished application only renders a constrained set of content specified by the developer, then the application might not be considered a user agent. In both cases, the WCAG 2.0 Guidelines apply to the web content. If the application is not a user agent, application developers are not responsible for UAAG 2.0 requirements that extend beyond WCAG 2.0 requirements. For more detail, see the definition of user agent.

Relationship to the Authoring Tool Accessibility Guidelines (ATAG) 2.0

While it is common to think of user agents retrieving and rendering web content for one group of people (end-users) that was previously authored by another group (authors), user agents are also frequently involved with the process of authoring content.

For these cases, it is important for user agent developers to consider the application of another W3C-WAI Recommendation, the [Authoring Tool Accessibility Guidelines \(ATAG\)](#). ATAG (currently 2.0 is in draft) provides guidance to the developers of tools regarding the accessibility of authoring interfaces to authors (ATAG 2.0 Part A) and ways in which all authors can be supported in producing accessible web content (ATAG 2.0 Part B).

For more information on the role of user agents in web authoring see [UAAG 2.0 Reference](#)

UAAG 2.0 Guidelines

The guidelines, success criteria, their notes, and the conformance applicability notes are normative. Guideline summaries are informative.

UAAG 2.0 Conformance Applicability Notes:

The Conformance Applicability Notes are a list of normative conditions that apply broadly to many of the success criteria in these guidelines. Generally, the notes clarify how the success criteria would apply under certain circumstances.

1. **Retrieved Content Only:** UAAG 2.0 success criteria only apply to web content that has been retrieved by the user agent (e.g. if a user agent saves bandwidth by

retrieving video element content only on demand, then captions associated with that video content do not need to be searchable as per 2.4.5 until the video is retrieved).

2. **Current Content Only:** At any point in time, UAAG 2.0 success criteria only apply to web content that has not been hidden or removed (e.g., a bookmark created as per [1.8.16](#) will no longer be operable if the content it refers to is hidden or removed).
 3. **Recognized Content Only:** UAAG 2.0 success criteria only apply to web content and its behaviors that can be recognized by user agents.
 4. **Optional Settings:** Throughout UAAG 2.0, all required behaviors can be provided as optional preference settings unless a success criterion explicitly says otherwise. For example, if a success criterion requires high contrast between foreground text and its background, the user agent can also provide choices with low contrast. While it is preferred to have a required behavior as a default option, it does not need to be, unless the success criterion explicitly says otherwise.
 5. **RFC 2119 language not used:** UAAG 2.0 does not use RFC 2119 language (must, may, should) as it is not an interoperable specifications. Note, even if these terms appear from time to time they do not have any RFC 2119 implication.
 6. **Simultaneous satisfaction of success criteria:** Users can access all behaviors required by UAAG 2.0 at the same time (e.g. when the user resizes the viewport per 1.8.8, content is reflowed per 1.8.6), except where those behaviors are mutually exclusive.
 7. **Vertical layout languages:** When user agents render vertical layout languages (e.g. Mongolian, Han), success criteria normally relating to horizontal rendering should be applied to vertical rendering instead.
 8. **Add-ons (Extensions and Plug-ins):** Success criteria can be met by a user agent alone or in conjunction with add-ons, as long as those are:
 - a. discoverable by the user
 - b. no extra cost to the user
 - c. easily installed (i.e. not requiring expert knowledge or editing of configuration files, databases, or registry entries)
- See [Components of UAAG 2.0 Conformance Claims](#).
9. **Relationship with operating system or platform:** The user agent does not need to implement every behavior itself. A required behavior can be provided by the platform, user agent, user agent add-ons, or potentially other layers. All are acceptable, as long as they are enumerated in the conformance claim.
 10. **Platform limitations:** If the platform (hardware or operating system) does not support a capability necessary for a given UAAG 2.0 success criterion, see [Components of UAAG 2.0 Conformance Claims](#) #8.
 11. **Override author settings for text configuration:** All of the success criteria under guideline 1.4 allow users to override the text characteristics specified by authors, and override user agent defaults.

PRINCIPLE 1 - Ensure that the user interface and rendered content are perceivable

Guideline 1.1 - Provide access to alternative content [[Reference for 1.1](#)]

Summary: The user can choose to render any type of alternative content available (1.1.1) with an indicator that the alternative content is present (1.1.2) or a placeholder replacing the non-text content (1.1.3) . It's recommended that users can also choose at least one alternative, such as alt text, to be displayed by default (1.1.5). It's recommended that caption text or sign language alternative

cannot obscure the video or the controls (1.1.4) and that the user can configure the text (1.1.6), size and position of media alternatives (1.1.7).

1.1.1 Render Alternative Content: The user can choose to render any type of recognized alternative content that is present for a content element. (Level A)

Note: It is recommended that the user agent allow the user to choose whether the alternative content replaces or supplements the original content element.

[Reference for 1.1.1](#)

1.1.2 Indicate Unrendered Alternative Content: The user can specify that indicators be displayed along with rendered content when recognized unrendered alternative content is present. (Level A)

[Reference for 1.1.2](#)

1.1.3 Replace Non-Text Content: The user can request a placeholder that incorporates recognized text alternative content instead of recognized non-text content, until explicit user request to render the non-text content. (Level A)

[Reference for 1.1.3](#)

1.1.4 Facilitate Clear Display of Alternative Content for Time-based Media: For recognized on-screen alternative content for time-based media (e.g. captions, sign language video), the following are all true: (Level A)

- **Don't obscure controls:** Displaying time-based media alternatives doesn't [obscure](#) recognized controls for the primary time-based media.
- **Don't obscure primary media:** The user can specify that displaying time-based media alternatives doesn't obscure the primary time-based media.

Note: Depending on the screen area available, the display of the primary time-based media can need to be reduced in size to meet this requirement.

[Reference for 1.1.4](#)

1.1.5 Provide Configurable Alternative Content Defaults: The user can specify which type(s) of alternative content to render by default for each type of non-text content, including time based media. (Level AA)

[Reference for 1.1.5](#)

1.1.6 Use Configurable Text for Time-based Media Captions: For recognized on-screen alternative content for time-based media (e.g. captions, sign language video), the user can configure recognized text within time-based media alternatives (e.g. captions) in conformance with [1.4.1](#). (Level AA)

[Reference for 1.1.6](#)

1.1.7 Allow Resize and Reposition of Time-based Media Alternatives: The user can configure recognized alternative content for time-based media (e.g. captions, sign language video) as follows: (Level AAA)

- **Resize:** The user can resize alternative content for time-based media to at least 50% of the size of the top-level viewports.
- **Reposition:** The user can reposition alternative content for time-based media to two or more of the following: above, below, to the right, to the left, and overlapping the primary time-based media.

Note 1: Depending on the screen area available, the display of the primary time-based media can need to be reduced in size or hidden to meet this requirement.

Note 2: Implementation can involve displaying alternative content for time-based media in a separate viewport, but this is not required.

[Reference for 1.1.7](#)

Guideline 1.2 - Repair missing content [\[Reference for 1.2\]](#)

Summary: The user can request useful alternative content when the author fails to provide it. For example, showing metadata in place of missing or empty (1.2.1) alt

text. The user can ask the browser to predict missing structural information, such as field labels, table headings or section headings (1.2.2).

1.2.1 Support Repair by Assistive Technologies: If text alternatives for non-text content are missing or empty, the user agent doesn't attempt to repair the text alternatives by substituting text values that are also available to assistive technologies (e.g. image file name). (Level AA)

[Reference for 1.2.1](#)

Guideline 1.3 - Provide highlighting for selection, keyboard focus, enabled elements, visited links [\[Reference for 1.3\]](#)

Summary: The user can visually distinguish between selected, focused, and enabled items; and recently visited links (1.3.1); with a choice of highlighting options that at least include foreground and background colors, and border color and thickness (1.3.2).

1.3.1 Distinguishable Highlighting: The user can have the following types of content uniquely highlighted, overriding any values specified by the author: (Level A)

- Selection
- In-page search results
- Active keyboard focus (indicated by focus cursors and/or text cursors)
- Unvisited links
- Visited links

[Reference for 1.3.1](#)

1.3.2 Highlighting Options: The user can set all of the following characteristics of selection highlighting, overriding any values specified by the author: (Level AA)

- Foreground color
- Background color

[Reference for 1.3.2](#)

1.3.3 Highlighting Active Keyboard Focus: The user can set all of the following characteristics of active keyboard focus highlighting, overriding any values specified by the author: (Level AA)

- Foreground color
- Background color
- Border (color, style, and thickness)
- Text cursor blink rate

[Reference for 1.3.3](#)

1.3.4 Distinguishing Enabled Elements: The user can set all of the following characteristics of enabled element highlighting, overriding any values specified by the author: (Level AA)

- Foreground color
- Background color
- Border (color, style, and thickness)

[Reference for 1.3.4](#)

1.3.5 Distinguishing Enabled Elements: The user can set all of the following characteristics for visited links and separately for unvisited links, overriding any values specified by the author: (Level AA)

- Foreground color
- Underline

[Reference for 1.3.5](#)

Guideline 1.4 - Provide text configuration [\[Reference for 1.4\]](#)

Summary: The user can set text scale, color, style, line spacing, and font family globally (1.4.1, Level A). It is recommended that the user agent implement the user-selected text configuration settings of the platform (1.4.5 Level AA), users set text size, color, line spacing, text style and font family for element types (1.4.2, Level AA); set character spacing, justification and margin sizes globally (1.4.3, Level AA); set capitalization, hyphenation, and borders globally (1.4.6, Level AAA); and print configured and reflowed text (1.4.4 Level AA).

Note 1: The success criteria in guideline 1.4 can be met through user stylesheets. For platforms without user stylesheets, text configuration needs to be provided to users through the user agent's main user interface or via an add-on.

Note 2: Users have varying needs for text size and spacing. Therefore, it's recommended that user agents provide a wider range of values, and a greater number of increments, to allow the user to adjust the view for their current task.

1.4.1 Basic text formatting (Globally): The user can globally set all of the following characteristics of visually rendered text content: (Level A)

- Text scale with preserved size distinctions (e.g. keeping headings proportional to main font)
- Text color and background color, choosing from all platform color options
- Font family, choosing from all installed fonts
- Line spacing, choosing from a range with at least three values up to at least 2 times the default
- Text style, choosing to turn on/off underline, italic, bold

[Reference for 1.4.1](#)

1.4.2 Basic text formatting (by Element): The user can set all of the following characteristics of visually rendered text content for text element types including at least headings, input fields, and links: (Level AA)

- Text size (e.g. 18 point) or scale (e.g. 150%)
- Text color and background color, choosing from all platform color options
- Font family, choosing from at least all installed fonts
- Line spacing, choosing from a range with at least three values up to at least 2 times the default
- Text style, choosing to turn on/off underline, italic, bold
- Margins around blocks of text
- Borders

[Reference for 1.4.2](#)

1.4.3 Blocks of text (Globally): The user can globally set all of the following characteristics of visually rendered blocks of text: (Level AA)

- Character spacing, choosing from a range with at least 5 values
- Justification (left or right, including turning off full justification)
- Margins around blocks of text
- Borders

[Reference for 1.4.3](#)

1.4.4 Configured and Reflowed Text Printing: The user can print the rendered content, and the following are all true: (Level AA)

- any visual, non-time-based, rendered content can be printed
- the user can choose between available printing devices
- the user can have content printed as it is rendered on screen, reflecting any user scaling, highlighting, and other modifications
- the user can have printed content reflow as if the top-level viewports had been resized to match the horizontal dimension of the printing device's printable area

[Reference for 1.4.4](#)

1.4.5 Default to platform text settings: The user can specify that platform text settings be used as the default values for text configuration. (Level AA)

[Reference for 1.4.5](#)

1.4.6 Advanced text formatting: The user can globally set all of the following characteristics of visually rendered blocks of text: (Level AAA)

- Capitalization (overriding upper case and small caps style)
- Word-breaking properties (e.g. auto-hyphenation)
- Word spacing (choosing from a range of at least 5 values)

Note: This success criterion does not apply to text entered as all caps. Content authors are encouraged to use styles instead of typing text as all caps.

[Reference for 1.4.6](#)

Guideline 1.5 - Provide volume configuration [\[Reference for 1.5\]](#)

Summary: The user can adjust the volume of each audio track relative to the global volume level (1.5.1).

1.5.1 Global Volume: The user can adjust the volume of each audio track independently of other tracks, relative to the global volume level set through operating environment mechanisms. (Level A)

[Reference for 1.5.1](#)

Guideline 1.6 - Provide synthesized speech configuration [\[Reference for 1.6\]](#)

Summary: If synthesized speech is produced, the user can specify speech rate, volume, and voice (1.6.1, Level A), pitch and pitch range (1.6.2, Level AA), advanced synthesizer speech characteristics such as emphasis (1.6.3, Level AAA) and features such as spelling (1.6.3, Level AAA).

Note: If browsers provide speech output for mainstream users, they should make the speech configurable enough to be usable by a wide range of individuals. When an add-on adds speech output to the user agent, it becomes part of the user agent, and therefore should meet the requirements of 1.6.

1.6.1 Speech Rate, Volume, and Voice: If synthesized speech is produced, the user can specify the following: (Level A)

- Speech rate
- Speech volume (independently of other sources of audio)
- Voice, when more than one voice is available

[Reference for 1.6.1](#)

1.6.2 Speech Pitch and Range: If synthesized speech is produced, the user can specify the following if offered by the speech synthesizer: (Level AA)

- Pitch (average frequency of the speaking voice)
- Pitch range (variation in average frequency)

Note: Because the technical implementations of text to speech engines vary (e.g. formant-based synthesis, concatenative synthesis), a specific engine may not support varying pitch or pitch range. A user agent should expose the availability of pitch and pitch range control if the currently selected or installed text to speech engine offers this capability.

[Reference for 1.6.2](#)

1.6.3 Synthesized Speech Features: If synthesized speech is produced, the following features are provided: (Level AA)

- **User-defined add-ons** to the synthesized speech dictionary.
- **"Spell-out"**: text is spelled one character at a time, or according to language-dependent pronunciation rules.
- **At least two ways of speaking numerals**: spoken as individual digits and punctuation (e.g. "one two zero three point five" for 1203.5 or "one comma two zero three point five" for 1,203.5), and spoken as full numbers are spoken (e.g. "one thousand, two hundred and three point five" for 1203.5).
- **At least two ways of speaking punctuation**: spoken literally, and with punctuation understood from speech characteristics like pauses.

[Reference for 1.6.3](#)

1.6.4 Synthesized Speech Language: If synthesized speech is produced and more than one language is available, the user can change the language. (Level AA)

[Reference for 1.6.4](#)

1.6.5 Advanced Speech Characteristics: If synthesized speech is produced, the user can adjust all of the speech characteristics provided by the speech synthesizer. (Level AAA)

[Reference for 1.6.5](#)

Guideline 1.7 - Enable configuration of user stylesheets [\[Reference for 1.7\]](#)

Summary: The user agent can disable author stylesheets (1.7.1, Level A) , supports user stylesheets or style mechanism (1.7.2, Level A), the user can choose which if any user-supplied (1.7.3, Level A) stylesheets to use, and the user can save stylesheets (1.7.4, Level AA).

1.7.1 Disable Author Stylesheets: If the user agent supports a mechanism for author styles, the user can disable the use of author styles on the current page. (Level A)

[Reference for 1.7.1](#)

1.7.2 Support User Stylesheet or User Style Modification Mechanism: If the user agent supports a mechanism for author styles, the user agent also provides a mechanism for a user styling to override author styling. (Level A)

[Reference for 1.7.2](#)

1.7.3 Apply User Stylesheets: If user styles are supported, then the user can enable or disable user styles for: (Level A)

- All pages on specified websites, or
- All pages

[Reference for 1.7.3](#)

1.7.4 Save Copies of Stylesheets: The user can save copies of the stylesheets referenced by the current page. This allows the user to edit and load the copies as user stylesheets. (Level AA)

[Reference for 1.7.4](#)

Guideline 1.8 - Help users to orient within, and control, windows and viewports [\[Reference for 1.8\]](#)

Summary: The user agent provides programmatic and visual cues to keep the user oriented. These include highlighting the viewport (1.8.1, Level A) and customizing the highlighting attributes (1.8.7, Level AA), keeping the focus within

the viewport (1.8.2 & 1.8.6, Level A), resizing the viewport (1.8.8, Level A), providing scrollbars that identify when content is outside the visible region (1.8.3, Level A) and which portion is visible (1.8.4, Level A), changing the size of graphical content with zoom (1.8.5, Level A & 1.8.7, Level A), and restoring the focus and point of regard when the user returns to a previously viewed page (1.8.9, Level AA). The user can specify that all viewports have the same user interface elements (1.8.12, Level AA), if and how new viewports open (1.8.10, Level AA), and whether the new viewport automatically gets focus (1.8.11, Level AA). The user can specify that multi-column text blocks be reflowed into a single column (1.8.13, Level AA), that the user can override absolute layout dimensions (1.8.14, Level AA), and linearize the content (1.8.15, Level AA). The user can mark items in a web page and use shortcuts to navigate back to marked items. (1.8.16, Level AAA).

1.8.1 Highlight Viewport: The user can have the viewport with the input focus be highlighted. (Level A)

[Reference for 1.8.1](#)

1.8.2 Move Viewport to Selection and Focus: When a viewport's selection or input focus changes, the viewport's content moves as necessary to ensure that the new selection or input focus location is at least partially in the visible portion of the viewport. (Level A)

[Reference for 1.8.2](#)

1.8.3 Provide Viewport Scrollbars: When the rendered content extends beyond the viewport dimensions, users can have graphical viewports include scrollbars, overriding any values specified by the author. (Level A)

[Reference for 1.8.3](#)

1.8.4 Indicate Viewport Position: The user can determine the viewport's position relative to the full extent of the rendered content. (Level A)

[Reference for 1.8.4](#)

1.8.5 Allow Zoom: The user can rescale content within top-level graphical viewports as follows: (Level A)

- **Zoom in:** to 500% or more of the default size
- **Zoom out:** to 10% or less of the default size, so the content fits within the height or width of the viewport

[Reference for 1.8.5](#)

1.8.6 Maintain Point of Regard: The point of regard remains visible within the viewport when the viewport is resized, when content is zoomed or scaled, or when content formatting is changed. (Level A)

Note: When the point of regard is larger than the viewport, the user agent keeps visible the beginning of the point of regard according to the current language's reading order (e.g. top-left in English)

[Reference for 1.8.6](#)

1.8.7 Customize Viewport Highlighting: When highlighting viewports as specified by [1.8.1 Highlight Viewport](#), the user can customize attributes of the viewport highlighting mechanism (e.g. color and width of borders). (Level AA)

[Reference for 1.8.7](#)

1.8.8 Allow Viewport Resize: The user can resize viewports within restrictions imposed by the platform, overriding any values specified by the [author](#). (Level AA)

[Reference for 1.8.8](#)

1.8.9 Provide Viewport History: For user agents that implement a history mechanism for top-level viewports (e.g. "back" button), the user can return to any state in the viewport history that is allowed by the content, including: (Level AA)

- a. restored point of regard
- b. input focus, and
- c. user's form field entries

Note: It is recommended that selection also be restored.

[Reference for 1.8.9](#)

1.8.10 Allow Top-Level Viewport Open on Request: The user can specify whether author content can open new top-level viewports (e.g. windows or tabs). (Level AA)

[Reference for 1.8.10](#)

1.8.11 Allow Top-Level Viewport Focus Control: If new top-level viewports (e.g. windows or tabs) are configured to open without explicit user request, the user can specify whether or not top-level viewports take the active keyboard focus when they open. (Level AA)

[Reference for 1.8.11](#)

1.8.12 Allow Same User Interface: The user can specify that all top-level viewports (e.g. windows or tabs) follow the defined user interface configuration. (Level AA)

[Reference for 1.8.12](#)

1.8.13 Multi-Column Text Reflow: The user can specify that recognized multi-column text blocks each be reflowed into a single column. (Level AA)

Note: Some layouts may become unusable if author-specified layout is overridden. In this case, the user can turn linearization off and try another strategy. It is recommended that user agents provide a convenient way for the user to turn this behavior on and off.

[Reference for 1.8.13](#)

1.8.14 Ignore Fixed Unit Dimensions: The user can have the user agent override author-specified unit dimensions. (Level AA)

[Reference for 1.8.14](#)

1.8.15 Linearize Content: The user can have recognized content rendered as a single column, overriding author-specified formatting of columns, tables, and positioning. (Level AA)

Note: Some layouts may become unusable if author-specified layout is overridden. In this case, the user can turn linearization off and try another strategy. It is recommended that user agents provide a convenient way for the user to turn this behavior on and off.

[Reference for 1.8.15](#)

1.8.16 Provide Web Page Bookmarks: The user can mark items in a web page, then use shortcuts to navigate back to marked items. The user can specify whether a navigation mark disappears after a session, or is persistent across sessions. (Level AAA)

[Reference for 1.8.16](#)

Guideline 1.9 - Provide alternative views [\[Reference for 1.9\]](#)

Summary: The user can view the source of content (1.9.2, Level AAA), and an outline view of content. (1.9.1, Level AA).

1.9.1 Outline View: Users can view a navigable outline of the headings in rendered content that allows focus to be moved to the corresponding element in the main viewport. (Level AA)

Note: Note: An outline view might also include other named elements such as document landmarks.

[Reference for 1.9.1](#)

1.9.2 Source View: The user can view all source text that is available to the user agent. (Level AAA)

[Reference for 1.9.2](#)

Guideline 1.10 - Provide element information [\[Reference for 1.10\]](#)

Summary: The user can access information about relationships between elements (e.g. form labels, table headers) (1.10.1, Level AA), and extended link information (e.g. title, internal vs. external) (1.10.2, Level AAA)

1.10.1 Show Related Elements: The user can access the information from explicitly-defined relationships in the content, including at least the following: (Level AA)

- a. calculated accessible name for images
- b. calculated accessible name for controls (e.g. form fields, buttons)
- c. caption for a table
- d. row and column labels for a table cell

[Reference for 1.10.1](#)

1.10.2 Show Element Hierarchy: The user can determine the path of element nodes going from the root element of the element hierarchy to the currently focused or selected element. (Level AAA)

[Reference for 1.10.2](#)

PRINCIPLE 2. Ensure that the user interface is operable

*Note: **Modality Independence:*** Users interacting with a web browser may do so using one or more input methods including keyboard, mouse, speech, touch, and gesture. It's critical that each user be free to use whatever input method, or combination of methods, works best for a given situation. If every potential user task is made accessible — so multiple modalities are supported — a user can choose what works best. For instance, if a user can't use or doesn't have access to a mouse, but can use and access a keyboard, the keyboard can call a modality independent control to activate an OnMouseOver event. Another example is a user on a mobile device that lacks keyboard who uses taps, wirelessly connected devices, and voice commands to simulate discrete or keyboard input. See [Independent User Interface: Events](#) for additional information on APIs and techniques for modality independent controls.

Guideline 2.1 - Ensure full keyboard access [\[Reference for 2.1\]](#)

Summary: Every viewport has a keyboard focus (2.1.2, Level A). Users can operate all functions using just the keyboard (2.1.1, Level A), activate important or common features with shortcut keys, (2.1.6, Level A), escape keyboard traps (2.1.3, Level A), specify that selecting an item in a dropdown list or menu not activate that item (2.1.4, Level A) and use standard keys for its platform (2.1.5, Level A).

2.1.1 Provide Full Keyboard Functionality: All functionality can be operated via the keyboard using sequential or direct keyboard commands that do not require specific timings for individual keystrokes, except where the underlying function requires input that depends on the path of the user's movement and not just the endpoints (e.g. free hand drawing). This does not forbid and should not discourage providing other input methods in addition to keyboard operation including mouse, touch, gesture and speech. (Level A)

[Reference for 2.1.1](#)

2.1.2 Has Keyboard Focus: Every viewport has an active or inactive keyboard focus at all times. (Level A)

[Reference for 2.1.2](#)

2.1.3 Avoid Keyboard Traps: If keyboard focus can be moved to a component using a keyboard interface (including nested user agents), then focus can be moved away from that component using only a keyboard interface. If this requires more than unmodified arrow or Tab keys (or standard exit methods like Escape), users are advised of the method for moving focus away. (Level A)

[Reference for 2.1.3](#)

2.1.4 Separate Selection from Activation: The user can specify that focus and selection can be moved without the user agent or author-supplied content further changing focus, selection, or the state of controls. (Level A)

[Reference for 2.1.4](#)

2.1.5 Follow Text Keyboard Conventions: The user agent follows keyboard conventions for the operating environment. (Level A)

[Reference for 2.1.5](#)

2.1.6 Make Keyboard Access Efficient: The user agent user interface includes mechanisms to make keyboard access more efficient than sequential keyboard access. (Level A)

[Reference for 2.1.6](#)

Guideline 2.2 - Provide sequential navigation [\[Reference for 2.2\]](#)

Summary: Users can use the keyboard to navigate sequentially to all the operable elements in the viewport (2.2.1, Level A) as well as between viewports (2.2.2, Level A), and the default navigation order is the document order (2.2.3, Level A). Users can optionally disable wrapping or request a signal when wrapping occurs (2.2.4, Level AA).

2.2.1 Sequential Navigation Between Elements: The user can move the keyboard focus backwards and forwards through all recognized enabled elements in the rendered content of the current top-level viewports. (Level A)

[Reference for 2.2.1](#)

2.2.2 Sequential Navigation Between Landmarks: The user can move the keyboard focus backwards and forwards between regions identified by document landmarks. (Level A)

Note: The user agent might also include other regions, such as viewports, in the sequential navigation.

[Reference for 2.2.2](#)

2.2.3 Default Navigation Order: If the author has not specified a navigation order, the user can have the default sequential navigation order be the source order. (Level AA)

[Reference for 2.2.3](#)

2.2.4 Options for Wrapping in Navigation: The user can request notification when sequential navigation wraps at the beginning or end of a document, and can prevent such wrapping. (Level AA)

[Reference for 2.2.4](#)

Guideline 2.3 - Provide direct navigation and activation [\[Reference for 2.3\]](#)

Summary: Users can navigate directly (e.g. using keyboard shortcuts) to elements (2.3.1, Level AA) with the option to immediately activate operable elements (2.3.2, Level AA). Display commands with the elements to make it easier for users to discover the commands (2.3.3 & 2.3.4, Level AA). The user can remap and save direct commands (2.3.5, Level AA).

2.3.1 Allow Direct Navigation to Enabled Elements: The user can move keyboard focus directly to any enabled element in the rendered content. (Level AA)

[Reference for 2.3.1](#)

2.3.2 Allow Direct Activation of Enabled Elements: The user can, in a single action, move keyboard focus directly to any enabled element in the rendered content and perform an activation action on that element. (Level AA)

[Reference for 2.3.2](#)

2.3.3 Present Direct Commands from Rendered Content: The user can have any recognized direct commands in rendered content (e.g. accesskey, landmark) be presented with their associated elements (e.g. Alt+R to reply to a web email). (Level AA)

[Reference for 2.3.3](#)

2.3.4 Present Direct Commands in User Interface: The user can have any direct commands in the UA user interface (e.g. keyboard shortcuts) be presented with their associated user interface controls (e.g. "Ctrl+S" displayed on the "Save" menu item and toolbar button). (Level AA)

[Reference for 2.3.4](#)

2.3.5 Allow Customized Keyboard Commands: The user can remap any keyboard shortcut including recognized author supplied shortcuts (e.g. accesskeys) and UA user interface controls, except for conventional bindings for the operating environment (e.g. arrow keys for navigating within menus). (Level AA)

[Reference for 2.3.5](#)

Guideline 2.4 - Provide text search [\[Reference for 2.4\]](#)

Summary: Users can search rendered content (2.4.1, Level A) forward or backward (2.4.2, Level A) and can have the matched content highlighted in the viewport (2.4.3, Level A). The user is notified in an accessible manner if there is no match (2.4.4, Level A). Users can also search by case and for text within alternative content (2.4.5, Level AA).

2.4.1 Text Search: The user can perform a search within rendered content, including rendered text alternatives and rendered generated content, for any sequence of printing characters from the document character set. (Level A)

[Reference for 2.4.1](#)

2.4.2 Search Direction: The user can search forward or backward in rendered content. (Level A)

[Reference for 2.4.2](#)

2.4.3 Match Found: When a search operation produces a match, the matched content is highlighted, the viewport is scrolled if necessary so that the matched content is within its visible area, and the user can search from the location of the match. (Level A)

[Reference for 2.4.3](#)

2.4.4 Alert on Wrap or No Match: The user can choose to receive notification when there is no match to a search operation. The user can choose to receive notification when the search continues from the beginning or end of content. (Level A)

[Reference for 2.4.4](#)

2.4.5 Alternative Content Search: The user can perform text searches within alternative content that is text (e.g. text alternatives for non-text content, captions) even when the alternative content is not rendered onscreen. (Level AA)

[Reference for 2.4.5](#)

Guideline 2.5 - Provide structural navigation [\[Reference for 2.5\]](#)

Summary: Users can navigate (2.5.1, Level A) content hierarchy.

2.5.1 Provide Structural Navigation by Heading and within Tables: The user agent provides at least the following types of structural navigation, where the structure types are recognized: (Level AA)

- By heading
- By content sections
- Within tables

[Reference for 2.5.1](#)

Guideline 2.6 - Configure and store preference settings [\[Reference for 2.6\]](#)

Summary: Users can restore preference settings to default (2.6.2, Level A), and accessibility settings persist between sessions (2.6.1, Level A). Users can manage multiple sets of preference settings (2.6.3, Level AA), and adjust preference settings outside the user interface so the current user interface does not prevent access (2.6.4, Level AA), and transport settings to compatible systems (2.6.5, Level AA).

2.6.1 Allow Persistent Accessibility Settings: User agent accessibility preference settings persist between sessions. (Level A)

Note: User agents may have a public access setting that turns this off.

[Reference for 2.6.1](#)

2.6.2 Allow Restore All to Default: The user can restore all preference settings to default values. (Level A)

[Reference for 2.6.2](#)

2.6.3 Allow Multiple Sets of Preference Settings: The user can save and retrieve multiple sets of user agent preference settings. (Level AA)

[Reference for 2.6.3](#)

2.6.4 Allow Preference Changes from outside the User Interface: The user can adjust any preference settings required to meet the User Agent Accessibility Guidelines (UAAG) 2.0 from outside the UA user interface. (Level AAA)

[Reference for 2.6.4](#)

2.6.5 Make Preference Settings Transferable: The user can transfer all compatible user agent preference settings between devices. (Level AAA)

[Reference for 2.6.5](#)

Guideline 2.7 - Customize display of graphical controls [\[Reference for 2.7\]](#)

Summary: It's recommended that users can add, remove, reposition, and assign shortcuts to user agent controls, and restore them to their default settings (2.7.1, Level AA).

2.7.1 Customize Display of Controls for User Interface Commands, Functions, and Add-ons: The user can customize which user agent commands, functions, and add-ons are displayed within the user agent user interface as follows: (Level AA)

- **Show:** The user can choose to display any controls available within the user agent user interface, including user-installed add-ons. It is acceptable to limit the total number of controls that are displayed onscreen.
- **Simplify:** The user can simplify the default user interface by choosing to display only commands essential for basic operation (e.g. by hiding some controls).

- **Reposition:** The user can choose to reposition individual controls within containers (e.g. toolbars or tool palettes), as well as reposition the containers themselves to facilitate physical access (e.g. to minimize hand travel on touch screens, or to facilitate preferred hand access on handheld mobile devices).
- **Assign Activation Keystrokes or Gestures:** The user can choose to view, assign or change default keystrokes or gestures used to activate controls.
- **Reset:** The user has the option to reset the containers and controls to their default configuration.

[Reference for 2.7.1](#)

Guideline 2.8 - Allow time-independent interaction [[Reference for 2.8](#)]

Summary: Users can extend the time limits for user input when such limits are controllable by the user agent (2.8.1, Level A).

2.8.1 Adjustable Time Limits: The UA user interface does not include time limits or at least one of the following is true: (Level A)

- a. **Turn Off:** Users are allowed to turn off the time limit before encountering it; or
- b. **Adjust:** Users are allowed to adjust the time limit before encountering it over a wide range that is at least ten times the length of the default setting; or
- c. **Extend:** Users are warned before time expires and given at least 20 seconds to extend the time limit with a simple action (e.g. "press the space bar"), and users are allowed to extend the time limit at least ten times; or
- d. **Real-time Exception:** The time limit is a required part of a real-time event and no alternative to the time limit is possible; or
- e. **Essential Exception:** The time limit is essential and extending it would invalidate the activity; or
- f. **20 Hour Exception:** The time limit is longer than 20 hours.

[Reference for 2.8.1](#)

Guideline 2.9 - Help users avoid flashing that could cause seizures [[Reference for 2.9](#)]

Summary: To help users avoid seizures, the default configuration prevents the browser user interface from flashing more than three times a second above luminescence or color thresholds (2.9.1, Level A), or even below the thresholds (2.9.2, Level AAA).

2.9.1 Three Flashes or Below Threshold: In its default configuration, the user agent does not display any UA user interface components that flashes more than three times in any one-second period, unless the flash is below general flash and red flash thresholds. (Level A)

[Reference for 2.9.1](#)

2.9.2 Three Flashes: In its default configuration, the user agent does not display any UA user interface components that flashes more than three times in any one-second period (regardless of whether not the flash is below the general flash and red flash thresholds). (Level AAA)

[Reference for 2.9.2](#)

Guideline 2.10 - Provide control of time-based media [[Reference for 2.10](#)]

Summary: The user can present placeholders for time-based media (2.10.1, Level A) and executable regions (2.10.2, Level A), or block all executable content (2.10.3, Level A), adjust playback (2.10.4, Level A), stop/pause/resume (2.10.5, Level A), navigate by time (2.10.6, Level A) or semantic structures such as chapter (2.10.7, Level AA). It is recommended that the user can adjust contrast and brightness of visual time-based media (2.10.8, Level AAA). Enable or disable tracks is included in [1.1.1 Render Alternative Content](#).

2.10.1 Time-Based Media Load-Only: The user can override the play on load of recognized time-based media content such that the content is not played until explicit user request. (Level A)

[Reference for 2.10.1](#)

2.10.2 Execution Placeholder: The user can request a [placeholder](#) instead of executable content that would normally be contained within an on-screen area (e.g. Applet, Flash), until explicit user request to execute. (Level A)

[Reference for 2.10.2](#)

2.10.3 Execution Toggle: The user can turn on/off the execution of dynamic or executable content (e.g. Javascript, canvas, media). (Level A)

[Reference for 2.10.3](#)

2.10.4 Adjustable Playback Rate for Prerecorded Content: The user can adjust the playback rate of prerecorded time-based media content, such that all of the following are true: (Level AA)

- **Playback Rate:** The user can adjust the playback rate of the time-based media tracks to between 50% and 250% of real time.
- **Pitch:** Speech whose playback rate has been adjusted by the user maintains pitch in order to limit degradation of the speech quality.
- **Synchronization:** Audio and video tracks remain synchronized across this required range of playback rates.
- **Reset:** The user agent provides a function that resets the playback rate to normal (100%).

[Reference for 2.10.4](#)

2.10.5 Stop/Pause/Resume Time-Based Media: The user can stop, pause, and resume rendered audio and animation content (e.g video, animation, changing text) that lasts three or more seconds at the default playback rate. (Level A)

[Reference for 2.10.5](#)

2.10.6 Navigation of Time-Based Media by Time: If time-based media lasts three or more seconds at the default playback rate, the user can navigate it using a continuous scale and by relative time units. (Level A)

[Reference for 2.10.6](#)

2.10.7 Navigation of Time-Based Media by Semantics: The user can navigate by semantic structure within the time-based media, such as by chapters or scenes present in the media. (Level AA)

[Reference for 2.10.7](#)

2.10.8 Video Contrast and Brightness: Users can adjust the contrast and brightness of visual time-based media. (Level AAA)

[Reference for 2.10.8](#)

Guideline 2.11 - Support other input devices [\[Reference for 2.11\]](#)

Summary: User agents support platform text input devices including text input (2.11.1, Level AA).

2.11.1 Text Input With Any Device: If an input device is supported by the platform, all user agent functionality including text input can be operated using that device. (Level AA)

[Reference for 2.11.1](#)

PRINCIPLE 3: Ensure that the user interface is understandable

Guideline 3.1 - Help users avoid and correct mistakes [\[Reference for 3.1\]](#)

Summary: Users can undo text entry (3.1.1, Level A), avoid or undo settings changes (3.1.2, Level A), and receive indications of progress activity (3.1.3, Level A). It is recommended that users can have their text checked for spelling errors (3.1.4, Level AA), go back after navigating (3.1.5, Level AA), have form submissions require confirmation (3.1.6, Level AA), have auto-form fill of basic information (3.1.7, Level AA), and save form entry data with a local save (3.1.8, Level AA).

3.1.1 Text Entry Undo: The user can reverse recognized text entry actions prior to submission. (Level A)

Note: Submission can be triggered in many different ways, such as clicking a submit button, typing a key in a control with an onkeypress event, or by a script responding to a timer.

[Reference for 3.1.1](#)

3.1.2 Settings Changes can be Reversed or Confirmed: If the user agent provides mechanisms for changing its user interface settings, it either allows the user to reverse the setting changes, or the user agent can require user confirmation to proceed. (Level A)

[Reference for 3.1.2](#)

3.1.3 Retrieval Progress: By default, the user agent shows the state of content retrieval activity. (Level A)

[Reference for 3.1.3](#)

3.1.4 Spell Check: The user can have spelling assistance for editable text in rendered content. (Level AA)

[Reference for 3.1.4](#)

3.1.5 Back Button: The user can reverse recognized navigation between web addresses (e.g. standard "back button" functionality). (Level AA)

[Reference for 3.1.5](#)

3.1.6 Form Submission Confirm: The user can specify whether or not recognized form submissions must be confirmed. (Level AA)

[Reference for 3.1.6](#)

3.1.7 Form Auto-Fill: The user can have the following information stored and used to auto-fill form fields by request: (Level AA)

- a. user's name
- b. user's email address
- c. user's phone number

[Reference for 3.1.7](#)

3.1.8 Save Form Entries: If the user agent provides a feature to save local versions of web content, then any form fields the user has filled retain any entries in the saved

version. (Level AA)

[Reference for 3.1.8](#)

Guideline 3.2 - Document the user agent user interface including accessibility features [\[Reference for 3.2\]](#)

Summary: User documentation is available in an accessible format (3.2.1, Level A), it includes accessibility features (3.2.2, Level A), it documents all the user features (3.2.3, Level AA), it delineates differences between versions (3.2.4, Level AA), and provides a centralized view of conformance UAAG2.0 (3.2.5, Level AAA).

3.2.1 Accessible Documentation: Product documentation is available in a format that meets success criteria of WCAG 2.0 level "A" or greater. (Level A)

[Reference for 3.2.1](#)

3.2.2 Describe Accessibility Features: For each user agent feature that is used to meet UAAG 2.0, at least one of the following is true: (Level A)

- a. **Described in the Documentation:** Use of the feature is explained in the user agent's documentation; or
- b. **Described in the Interface:** Use of the feature is explained in the UA user interface; or
- c. **Platform Service:** The feature is a service provided by an underlying platform; or
- d. **Not Used by Users:** The feature is not used directly by users (e.g., passing information to a platform accessibility service).

[Reference for 3.2.2](#)

3.2.3 Document All Features: For each user agent feature, at least one of the following is true: (Level AA)

- a. **Described in the Documentation:** Use of the feature is explained in the user agent's documentation; or
- b. **Described in the Interface:** Use of the feature is explained in the UA user interface; or
- c. **Platform Service:** The feature is a service provided by an underlying platform; or
- d. **Not Used by Users:** The feature is not used directly by users (e.g., passing information to a platform accessibility service).

[Reference for 3.2.3](#)

3.2.4 Changes Between Versions: Changes to features that meet UAAG 2.0 success criteria since the previous user agent release are documented. (Level AA)

[Reference for 3.2.4](#)

3.2.5 Centralized View: There is a dedicated section of the documentation that presents a view of all features of the user agent necessary to meet the requirements of User Agent Accessibility Guidelines 2.0. (Level AAA)

[Reference for 3.2.5](#)

Guideline 3.3 - Make the user agent behave in predictable ways [\[Reference for 3.3\]](#)

Summary: Users can prevent non-requested focus changes (3.3.1, Level A).

3.3.1 Avoid Unpredictable Focus: The user can prevent focus changes that are not a result of explicit user request. (Level A)

[Reference for 3.3.1](#)

PRINCIPLE 4: Facilitate programmatic access

Guideline 4.1 - Facilitate programmatic access to assistive technology [\[Reference for 4.1\]](#)

Summary: The user agent supports platform accessibility services (4.1.1, Level A), including providing information about all controls and operation (4.1.2, Level A), access to DOMs when platform accessibility services are not available (4.1.4, Level A). Controls can be adjusted programmatically (4.1.5, Level A). Where something can't be made accessible, provide an accessible alternative version, such as a standard window in place of a customized window (4.1.3, Level A).

Note: UAAG 2.0 assumes that a platform accessibility service will be built on top of underlying security architectures that will allow user agents to comply with both the success criteria and security needs.

4.1.1 Support Platform Accessibility Services: The user agent supports relevant platform accessibility services. (Level A)

[Reference for 4.1.1](#)

4.1.2 Expose Accessible Properties: For all user interface components (including UA user interface, rendered content, and generated content) the user agent makes available the following properties and any change notifications via a platform accessibility service: (Level A)

- Name, Role, State
- Value
- Selection
- Focus
- Bounding dimensions and coordinates
- Font family of text
- Foreground and background color for text
- Highlighting
- Keyboard commands
- Caret position
- Explicitly defined relationships (e.g. ARIA relationships [\[ARIA 1.0\]](#))

[Reference for 4.1.2](#)

4.1.3 Provide Equivalent Accessible Alternatives: If UA user interface functionality cannot be exposed through platform accessibility services, then the user agent provides equivalent functionality that can be exposed through the platform accessibility service. (Level A)

[Reference for 4.1.3](#)

4.1.4 DOMs Programmatically Available as fallback: If the user agent accessibility API does not provide sufficient information to one or more platform accessibility services, then Document Object Models (DOM), must be made programmatically available to assistive technologies. (Level A)

[Reference for 4.1.4](#)

4.1.5 Make Content Interaction Programmatically Available: If the user can interact with content (e.g. by checking a box or editing a text area), the same degree of interaction is programmatically available. (Level A)

[Reference for 4.1.5](#)

PRINCIPLE 5: Comply with applicable specifications and conventions

Guideline 5.1 - Comply with applicable specifications and conventions [[Reference for 5.1](#)]

Summary: When the browser's controls are authored in HTML or similar standards, they need to meet W3C's Web Content Accessibility Guidelines (5.1.1, Levels A, AA, AAA). The user agent supports the accessibility features of content formats (5.1.2, Level A) and of the platform (5.1.3, Level A), allows handling of unrendered technologies (5.1.4, Level A), allows alternative viewers (5.1.4, Level AA), and allows users to report accessibility issues (5.1.5, Level AAA).

5.1.1 Comply with WCAG: Web-based UA user interfaces meet the WCAG 2.0 success criteria. (Level A to meet WCAG 2.0 Level A success criteria; Level AA to meet WCAG 2.0 Level A and AA success criteria; and Level AAA to meet WCAG 2.0 Level A, AA, and AAA success criteria)

Note: This success criterion does not apply to native UA user interfaces, but does include any parts of native user agents that are web-based (e.g. help systems). However, it is recommended that developers of native user agent user interfaces follow the Guidance on Applying WCAG 2.0 to Non-Web Information and Communications Technologies (WCAG2ICT) [[WCAG2ICT](#)].

[[Reference for 5.1.1](#)]

5.1.2 Implement Accessibility Features of Web Content Technology Specifications:

Implement the accessibility features of web content technology specifications.

Accessibility features are those that are either (Level A):

- Identified as such in the content specifications or
- Allow authors to satisfy a requirement of WCAG 2.0

Note 1: If a conformance claim is filed, cite the implemented specifications in the conformance claim. See [Components of a UAAG 2.0 Conformance Claim](#) - #9 Web Content Technologies.

Note 2: When a rendering requirement of another specification contradicts a requirement of UAAG 2.0, the user agent can disregard the rendering requirement of the other specification and still satisfy this guideline.

[[Reference for 5.1.2](#)]

5.1.3 Implement Accessibility Features of the Platform: If the user agent contains native user interfaces, then those native user interfaces follow user interface accessibility guidelines for the platform. (Level A)

[[Reference for 5.1.3](#)]

5.1.4 Allow Content Elements to be Rendered in Alternative Viewers: The user can select content elements and have them rendered in alternative viewers. (Level AA)

[[Reference for 5.1.4](#)]

5.1.5 Enable Reporting of User Agent Accessibility Faults: The user agent provides a mechanism for users to report user agent accessibility issues. (Level AAA)

[[Reference for 5.1.5](#)]

Conformance

This section is normative.

Conformance means that the user agent satisfies the success criteria defined in the guidelines section. This section lists requirements for conformance and conformance

claims.

Conformance Requirements

In order for a web page to conform to UAAG 2.0, one of the following levels of conformance is met in full.

- Level A: For level A conformance (the minimum level of conformance), the user agent satisfies all the Level A success criteria.
- Level AA: For level AA conformance (recommended), the user agent satisfies all level A and level AA Success Criteria.
- Level AAA: For level AAA conformance (advanced), the user agent satisfies all level A, level AA and level AAA Success Criteria.

The [Conformance Applicability Notes](#) provide additional guidance on the applicability of the success criteria under certain circumstances.

Although conformance can only be achieved at the stated levels, developers are encouraged to report (in their claim) any progress toward meeting success criteria from all levels beyond the achieved level of conformance.

Conformance Claims

Conditions on Conformance Claims

If a conformance claim is made, the conformance claim must meet the following conditions:

- At least one version of the conformance claim must be published on the web as a document meeting level "A" of WCAG 2.0. A suggested metadata description for this document is "UAAG 2.0 Conformance Claim".
- Whenever the claimed conformance level is published (e.g. product information website), the URI for the on-line published version of the conformance claim must be included.
- The existence of a conformance claim does not imply that the W3C has reviewed the claim or assured its validity.
- There are no restrictions on who can make a claim.
- Claimants are solely responsible for the accuracy of their claims.
- Claimants are encouraged to claim conformance to the most recent version of the User Agent Accessibility Guidelines Recommendation.

Components of UAAG 2.0 Conformance Claims

1. Claimant name and affiliation
2. Claimant contact information
3. Date of the claim
4. Type of compliance: [] User Agent (full) [] [Add-on](#) only (limited)
5. Conformance level satisfied
6. **User agent information:**
 - a. Name and manufacturer
 - b. Version number or version range
 - c. Required patches or updates, language of the user interface and documentation (e.g. English, French, Chinese)
 - d. Plug-ins or add-ons (including version numbers) needed to meet the success criteria (e.g. mouseless browsing)
 - e. Configuration changes to the user agent, plug-ins and add-ons that are needed to meet the success criteria (e.g. ignore author)

- foreground/background color, turn on Carat Browsing)
7. **Platform:** Provide relevant information about the software and/or hardware platform(s) that the user agent relies on for conformance. This information can include:
 - a. Name and manufacturer
 - b. Version of key software components (e.g. operating system, other software environment)
 - c. Hardware requirements (e.g. audio output enabled, minimum screen size: 2", bluetooth keyboard attached)
 - d. Operating system(s) (e.g. Windows, Android, iOS, GNOME)
 - e. Other software environment (Java, Eclipse)
 - f. Host web browser when the conforming user agent is web-based (e.g. JW Player on Firefox)
 - g. Configuration changes to the platform that are needed to meet the success criteria (e.g. turn on Sticky Keys, use High Contrast Mode)
 8. **Platform Limitations:** If the platform (hardware or operating system) does not support a capability necessary for a given UAAG 2.0 success criterion, list the success criterion and the feature (e.g. a mobile operating system does not support platform accessibility services, therefore the user agent cannot meet success criterion 4.1.2). For these listed features, the user agent can claim that the success criteria do not apply (see 10.b.1 following).
 9. **Web Content Technologies:** List the web content technologies rendered by the user agent that are included in the claim. If there are any web content technologies rendered by the user agent that are excluded from the conformance claim, list these separately. Examples of web content technologies include web markup languages such HTML, XML, CSS, SVG, and MathML, image formats such as PNG, JPG and GIF, scripting languages such as JavaScript/EcmaScript, specific video codecs, and proprietary document formats.
 10. **Declarations:** For each success criterion, provide a declaration of either
 - a. whether or not the success criterion has been satisfied; or
 - b. declaration that the success criterion is not applicable and a rationale for why not, from the following choices:
 1. **Platform:** not applicable due constraints of the platform, per Paragraph 7 above (e.g. color handling on a monochrome device, video handling in a purely audio browser, or interprocess communication on an operating system that does not support multitasking). Describe the specific platform limitation.
 2. **Input:** not applicable due to a constrained input set (e.g. a help system that only displays the HTML files included with the product)
 3. **Output:** not applicable due to intentionally limited output modalities (e.g. video handling in a browser that only does audio output, even though the platform can support video)

Limited Conformance for Add-ons

This option can be used for a user agent add-on or plug-in with limited functionality that wishes to claim UAAG 2.0 conformance. An add-on or plug-in can claim conformance for a specific success criterion or a narrow range of success criteria as stated in the claim. All other success criteria can be denoted as Not Applicable.

UAAG recognizes that some add-ons can be so specialized to the needs of a particular disability that the add-on is mutually exclusive with other success criteria of UAAG, but the goal would be for add-ons to work with the user agent so that any features of the user agent needed for UAAG conformance are not broken by one add-on. If the add-on limits other accessibility features of the user agent, then include a statement to that effect, such as: "This add-on breaks success criterion x.x.x because it is intended to meet [foo] need of [this] class of user." An example would be a (hypothetical) add-on that breaks 1.8.2

and 1.8.3 (viewport navigation) to provide a simplified page for people with high distraction levels.

Optional Components of an UAAG 2.0 Conformance Claim

A description of how the UAAG 2.0 success criteria were met where this is not obvious.

Disclaimer

Neither W3C, WAI, nor UAWG take any responsibility for any aspect or result of any UAAG 2.0 conformance claim that has not been published under the authority of the W3C, WAI, or UAWG.

Appendix A: Glossary

This glossary is normative.

[a](#) · [b](#) · [c](#) · [d](#) · [e](#) · [f](#) · [g](#) · [h](#) · [i](#) · [j](#) · [k](#) · [l](#) · [m](#) · [n](#) · [o](#) · [p](#) · [q](#) · [r](#) · [s](#) · [t](#) · [u](#) · [v](#) · [w](#) · [x](#) · [y](#) · [z](#)

activate

To carry out the behaviors associated with an enabled element in the rendered content or a component of the UA user interface.

alternative content

Web content that user agents can programmatically determine is usable in place of other content that some people are not able to access. Alternative content fulfills essentially the same function or purpose as the original content. There are several general types of alternative content:

- **text alternative for non-text content:** Text that is programmatically associated with non-text content or referred to from text that is programmatically associated with non-text content. For example, an image of a chart might have two text alternatives: a short text alternative and a described-by relationship to a nearby paragraph that more fully describes the content of the chart.
- **alternative for time-based media:** Web content that serves the same function or purpose as one or more tracks in a time-based media presentation. This includes alternatives for audio such as captions and sign language interpretation, and alternatives for video such as audio descriptions and extended audio descriptions. Another form of time-based media alternative is a correctly sequenced text description of time-based visual and auditory information that also is capable of achieving the outcomes of any interactivity in the time-based presentation.
- **text alternatives for text content:** Text that is programmatically associated with certain types of text content to provide expanded information. For example, an abbreviation (or acronym) can provide an expansion of the shortened word or initialized words (`<abbr title="User Agent Accessibility Guidelines">UAAG</abbr>`).
- **media alternative for text:** Media that presents no more information than is already presented in text (directly or via text alternatives). A media alternative for text is provided for people who benefit from alternate representations of text. Media alternatives for text can be audio-only, video-only (including sign-language video), or audio-video.

Note: According to WCAG 2.0, alternative content may or may not be programmatically determinable (e.g., a short description for an image might appear in the image's description attribute or within text near the image). However, UAAG 2.0 adds the programmatically available condition because this is the only type of alternative content that user agents can recognize.

animation

Graphical [content](#) rendered to automatically change over time, giving the user a visual perception of movement. Examples include [video](#), animated images, scrolling text, programmatic animation (e.g. moving or replacing rendered objects).

application programming interface (API)

A mechanism that defines how communication can take place between applications.

assistive technology

For the purpose of UAAG 2.0 conformance, assistive technology meets the following criteria:

1. Relies on services (such as retrieving [web resources](#) and parsing markup) provided by one or more host user agents.
2. Communicates data and messages with host user agents by monitoring and using APIs.
3. Provides services beyond those offered by the host user agents to meet the requirements of users with disabilities. Additional services include alternative renderings (e.g. as synthesized speech or magnified content), alternative input methods (e.g. voice), additional navigation or orientation mechanisms, and content transformations (e.g. to make tables more accessible).

Examples of assistive technologies that are important in the context of UAAG 2.0 include the following:

- Screen magnifiers, which are used by people with visual disabilities to enlarge and change colors on the screen to improve the visual readability of rendered text and images.
- Screen readers, which are used by people who are blind or have reading disabilities to read textual information through synthesized speech or braille displays.
- Voice recognition software, which is used by some people who have physical disabilities to simulate the keyboard and mouse.
- Alternative keyboards, which are used by some people with physical disabilities to simulate the keyboard and mouse.
- Alternative pointing devices, which are used by some people with physical disabilities to simulate mouse pointing and button activations.

audio

The technology of sound transmission. Audio can be created synthetically (including speech synthesis), streamed from a live source (e.g. a radio broadcast), or recorded from real world sounds. There can be multiple [audio tracks](#) in a presentation.

audio description

A type of [alternative content](#) that takes the form of narration added to the [audio](#) to describe important visual details that cannot be understood from the main soundtrack alone. Audio description of [video](#) provides information about actions, characters, scene changes, on-screen text, and other visual content. In standard audio description, narration is added during existing pauses in dialogue.

- **extended audio description:** An [audio description](#) in which the [video](#) is sometimes paused so that there is time to add additional description.

audio track

All or part of the [audio](#) portion of a presentation (e.g. each instrument can have a track, or each stereo channel can have a track).

author

A person who works alone or collaboratively to create [content](#) (e.g. content author, designer, programmer, publisher, tester).

available printing devices

Printing devices that are identified as available to applications via the [platform](#).

captions

A type of [alternative content](#) that takes the form of text presented and synchronized with time-based media to provide not only the speech, but also non-speech information conveyed through sound, including meaningful sound effects and identification of speakers. In some countries, the term "subtitle" is used to refer to

dialogue only and "captions" is used as the term for dialogue plus sounds and speaker identification. In other countries, "subtitle" (or its translation) is used to refer to both.

- **open captions:** Captions that are always rendered with a visual track; they cannot be turned off.
- **closed captions:** Captions that can be turned on and off. The captions requirements of UAAG 2.0 assume that the user agent can recognize the captions as such.

Note: Other terms that include the word "caption" can have different meanings. For instance, a "table caption" is a title for a table, often positioned graphically above or below the table.

commands

Actions made by users to control the [user agent](#). These include:

- **direct commands:** Commands that apply to a specified item (e.g. button) or action (e.g. save function), regardless of the current focus location. Also see [keyboard command](#)
 - **direct navigation commands:** Commands that move focus to a specified item.
 - **direct activation commands:** Commands that activate the specified item (and can also move focus to it) or action.
- **sequential navigation commands** (sometimes called "logical navigation commands" or "linear navigation commands"): Commands that move focus forwards and backwards through a list of items. The element list being navigated can be the list of all elements or just a subset (e.g. the list of headers, the list of links).
- **spatial commands** (sometimes called "directional commands"): Commands that require the user to be aware of the spatial arrangement of items on the screen:
 - **spatial navigation commands:** Commands that move from one item to another based on direction on the screen.
 - **spatial manipulation commands:** Commands that resize or reposition an item on the screen.
- **structural navigation commands:** Commands that move forwards, backwards, up and down a hierarchy.

content (web content)

Information and sensory experience to be communicated to the user by means of a [user agent](#), including code or markup that defines the content's structure, presentation, and interactions.

continuous scale

When interacting with a time-based media presentation, a continuous scale allows user (or programmatic) action to set the active playback position to any time point on the presentation time line. The granularity of the positioning is determined by the smallest resolvable time unit in the media timebase.

default

see [properties](#)

directly

using a [direct command](#)

disabled element

see [element](#)

document character set

The internal representation of data in the [source](#) content by a [user agent](#).

document object, Document Object Model (DOM)

A platform- and language-neutral interface that allows programs and scripts to dynamically access and update the content, structure and style of documents. The document can be further processed and the results of that processing can be incorporated back into the presented page. Overview of DOM-related materials: <http://www.w3.org/DOM/#what>.

documentation

Any information that supports the use of a [user agent](#). This information can be provided electronically or otherwise and includes help, manuals, installation instructions, tutorials, etc. Documentation can be accessed in various ways (e.g. as files included in the installation, available on the web).

Note: The level of technical detail in documentation for users should match the technical level of the feature. For example, user documentation for a browser's zoom function should not refer users to the source code repository for that browser.

element

Primarily, a syntactic construct of a document type definition (DTD) for its application. This is the sense employed by the XML 1.0 specification ([\[XML\]](#), section 3). UAAG 2.0 also uses the term "element" more generally to refer to any discrete unit within the content (e.g. a specific image, video, sound, heading, list, or list item).

- **enabled element:** An [element](#) with associated behaviors that can be activated through the user interface or through an API. The set of elements that a user agent enables is generally derived from, but is not limited to, the set of elements defined by implemented markup languages.
- **disabled element:** A potentially enabled element that is not currently available for activation (e.g. a "grayed out" menu item).
- **element type:** A category of elements (such as images, videos, sounds, first level headings, lists, or list items).

events and scripting, event handler, event type

User agents often perform a task when an event having a particular "event type" occurs, including a user interface event, a change to content, loading of content, or a request from the [operating environment](#). Some markup languages allow authors to specify that a script, called an **event handler**, be executed when an event of a given type occurs. An event handler is **explicitly associated with an element** through scripting, markup or the [DOM](#).

enabled element

see [element](#)

explicit user request

An interaction by the user through the [UA user interface](#), the [focus](#), or the [selection](#). User requests are made, for example, through [user agent user interface controls](#) and [keyboard commands](#). Some examples of explicit user requests include when the user selects "New viewport," responds "yes" to a [prompt](#) in the user agent's user interface, configures the user agent to behave in a certain way, or changes the selection or focus with the keyboard or pointing device. *Note:* Users can make errors when interacting with the user agent. For example, a user can inadvertently respond "yes" to a prompt instead of "no." This type of error is still considered an explicit user request.

extended audio description

see [audio description](#)

focus, input focus

The location where input will occur if a [viewport](#) is active. Examples include:

- **keyboard focus:** The screen location where [keyboard](#) input will occur if a viewport is active.
- **pointing device focus:** The screen location where [pointer](#) input will occur if a viewport is active. There can be multiple pointing device foci for example when using a screen sharing utility there is typically one for the user's physical mouse and one for the remote mouse.

The **active input focus** is in the active viewport. The **inactive input focus** is in the inactive viewport. Focus is typically indicated by a [focus cursor](#).

focus cursor

Visual indicator that [highlights](#) a user interface element to show that it has [input focus](#) (e.g. the dotted line around a button, outline around a pane, or brightened title bar on a window).

- **keyboard focus cursor:** Indicator showing the enabled element with current input focus where focus has been moved by the keyboard (e.g. the dotted line

around a button).

- **text cursor:** Indicator showing where keyboard input will occur in text (e.g. the flashing vertical bar in a text field, also called a caret).
- **pointer:** Indicator showing where pointing device input will occur. The indicator can be moved with a pointing device or emulator such as a mouse, pen tablet, keyboard-based mouse emulator, speech-based mouse commands, or 3-D wand. A pointing device click typically moves the [input focus](#) to the pointer location. The indicator can change to reflect different states. When touchscreens are used, the "pointing device" is a combination of the touchscreen and the user's finger or stylus. On most touchscreen systems there is no pointer (on-screen visual indication).

Cursors are active when in the active viewport, and inactive when in an inactive viewport.

focusable element

Any [element](#) capable of having [input focus](#) (e.g. a link, text box, or menu item). In order to be accessible and fully usable, every focusable element should take [keyboard focus](#), and ideally would also take [pointer focus](#).

globally, global configuration

A setting is one that applies to the entire [user agent](#) or all content being rendered by it, rather than to a specific feature within the user agent or a specific document being viewed.

graphical

Information (e.g. text, colors, graphics, images, or [animations](#)) rendered for visual consumption.

highlight, highlighted, highlighting

Emphasis indicated through the user interface. For example, [user agents](#) highlight content that is selected, focused, or matched by a search operation. Graphical highlight mechanisms include dotted boxes, changed colors or fonts, underlining, adjacent icons, magnification, and reverse video. Synthesized speech highlight mechanisms include alterations of voice pitch and volume (i.e. speech prosody). User interface items can also be highlighted, for example a specific set of foreground and background colors for the title bar of the active window. Content that is highlighted may or may not be a [selection](#).

image

Pictorial content that is static (i.e. not moving or changing). Also see [animation](#).

informative (non-normative)

see [normative](#)

keyboard

The letter, symbol and command keys or key indicators that allow a user to control a computing device. Assistive technologies have traditionally relied on the keyboard interface as a universal, or modality independent interface. In this document references to keyboard include keyboard emulators and [keyboard interfaces](#) that make use of the keyboard's role as a modality independent interface (see [Modality Independence](#)). Keyboard emulators and interfaces can be used on devices which do not have a physical keyboard, such as mobile devices based on touchscreen input.

keyboard interface

Keyboard interfaces are programmatic services provided by many platforms that allow operation in a device independent manner. A keyboard interface can allow keystroke input even if particular devices do not contain a hardware keyboard (e.g. a touchscreen-controlled device can have a keyboard interface built into its [operating system](#) to support onscreen keyboards as well as external keyboards that can be connected).

Note: Keyboard-operated mouse emulators, such as MouseKeys, do not qualify as operation through a keyboard interface because these emulators use pointing device interfaces, not keyboard interfaces.

keyboard command (keyboard binding, keyboard shortcuts, accesskey, access key, accelerator keys, direct keyboard command)

A key or set of keys that are tied to a particular UI control or application function, allowing the user to navigate to or activate the control or function without traversing any intervening controls (e.g. CTRL+"S" to save a document). It is sometimes useful to distinguish keyboard commands that are associated with controls that are rendered in the current context (e.g. ALT+"D" to move focus to the address bar) from those that can be able to activate program functionality that is not associated with any currently rendered controls (e.g. "F1" to open the Help system). Keyboard commands can be triggered using a physical keyboard or keyboard emulator (e.g. on-screen keyboard or speech recognition). (See [Modality Independent Controls](#)).

Sequential keyboard commands require multiple keystrokes to carry out an action (e.g. a series of Tab or arrow presses followed by Enter, or a sequence like ALT-F, V to drop down a File menu and choose Print Preview).

non-text content (non-text element, non-text equivalent)

see [text](#)

normative, informative (non-normative)

Required (or not required) for conformance. Abilities identified as "normative" are required for [conformance](#) (noting that one can conform in a variety of well-defined ways to UAAG 2.0). Abilities identified as "informative" (or, "non-normative") are never required for conformance.

notify

To make the user aware of events or status changes. Notifications can occur within the [UA user interface](#) (e.g. a status bar) or within the content display. Notifications can be passive and not require user acknowledgment, or they can be presented in the form of a [prompt](#) requesting a user response (e.g. a confirmation dialog).

obscure

To render a visual element in the same screen space as a second visual element in a way that prevents the second visual element from being visually perceived.

Note: The use of transparent backgrounds for the overlaying visual element (e.g., video captions) is an acceptable technique for reducing obscuration, if space is available.

operating environment

The software environment that governs the user agent's operation, whether it is an operating system or a programming language environment such as Java.

operating system (OS)

Software that supports a device's basic functions, such as scheduling tasks, executing applications, and managing hardware and peripherals.

Note: Many operating systems mediate communication between executing applications and [assistive technology](#) via a [platform accessibility service](#).

override

When one configuration or behavior preference prevails over another. Generally, the requirements of UAAG 2.0 involve user preferences prevailing over author preferences and user agent default settings and behaviors. Preferences can be multi-valued in general (e.g. the user prefers blue over red or yellow), and include the special case of two values (e.g. turn on or off blinking text content).

placeholder

Content generated by the user agent to replace author-supplied content. A placeholder can be generated as the result of a user preference (e.g. to not render images) or as [repair content](#) (e.g. when an image cannot be found). A placeholder can be any type of content, including text, images, and audio cues. A placeholder should identify the technology of the replaced object.

platform

The software and hardware environment(s) within which the [user agent](#) operates. Platforms provide a consistent operational environment. There can be layers of software in an hardware architecture and each layer can be considered a platform. Native platforms include [desktop operating system](#) (e.g. Linux, Mac OS, Windows, etc.), mobile operating systems (e.g. Android, BlackBerry, iOS, Windows Phone, etc.), and cross-OS environments (e.g. Java). Web-based platforms are other user agents. User agents can employ server-based processing, such as web content

transformations, text-to-speech production, etc.

Note 1: A user agent can include functionality hosted on multiple platforms (e.g. a browser running on the desktop can include server-based pre-processing and web-based documentation).

Note 2: Accessibility guidelines for developers exist for many platforms.

platform accessibility service

A programmatic interface that is engineered to enhance communication between mainstream software applications and assistive technologies (e.g. MSAA, UI Automation, and IAccessible2 for Windows applications, AXAPI for Mac OSX applications, Gnome Accessibility Toolkit API for GNOME applications, Java Access for Java applications). On some platforms it can be conventional to enhance communication further by implementing a DOM.

plug-in

see [user agent](#)

point of regard

The position in rendered content that the user is presumed to be viewing. The dimensions of the point of regard can vary. For example, it can be a two-dimensional area (e.g. content rendered through a two-dimensional graphical viewport), or a point (e.g. a moment during an audio rendering or a cursor position in a graphical rendering), or a range of text (e.g. focused text), or a two-dimensional area (e.g. content rendered through a two-dimensional graphical viewport). The point of regard is almost always within the viewport, but it can exceed the spatial or temporal [dimensions](#) of the viewport (see the definition of rendered content for more information about viewport dimensions). The point of regard can also refer to a particular moment in time for content that changes over time (e.g. an audio-only presentation). User agents can determine the point of regard in a number of ways, including based on viewport position in content, keyboard focus, and selection.

pointer

see [focus cursor](#)

profile

A named and persistent representation of user preferences that can be used to configure a [user agent](#). Preferences include input configurations, style preferences, and natural language preferences. In operating environments with distinct user accounts, profiles enable users to reconfigure software quickly when they log on. Users can share their profiles with one another. Platform-independent profiles are useful for those who use the same user agent on different devices.

programmatically available

Information that is encoded in a way that allows different software, including [assistive technologies](#), to extract and use the information relying on published, supported mechanisms, such as, platform accessibility services, APIs, or the document object models (DOM). For web-based user interfaces, this means ensuring that the [user agent](#) can pass on the information (e.g. through the use of WAI-ARIA). Something is programmatically available if the entity presenting the information does so in a way that is explicit and unambiguous, in a way that can be understood without reverse-engineering or complex (and thus potentially fallible) heuristics, and only relying on methods that are published, and officially supported by the developers of the software being evaluated.

prompt

Any [user agent](#)-initiated request for a decision or piece of information from a user.

properties, values, and defaults

A user agent renders a document by applying formatting algorithms and style information to the document's elements. Formatting depends on a number of factors, including where the document is rendered (e.g. on screen, on paper, through loudspeakers, on a braille display, on a mobile device). Style information (e.g. fonts, colors, synthesized speech prosody) can come from the elements themselves (e.g. certain font and phrase elements in HTML), from stylesheets, or from user agent settings. For the purposes of these guidelines, each formatting or style option is governed by a property and each property can take one value from a

set of legal values. Generally in UAAG 2.0, the term "[property](#)" has the meaning defined in [CSS 2.1 Conformance](#) ([\[CSS21\]](#),). A reference to "styles" in UAAG 2.0 means a set of style-related properties.

- **default value:** The value given to a property by a user agent at installation.

recognize

Information or events that can be identified unambiguously by user agents.

recognized content: Information that is encoded within content in a way that can be unambiguously recognized by user agents. [Authors](#) encode information in many ways, including in markup languages, style sheet languages, scripting languages, and protocols. When the information is encoded in a manner that allows the user agent to process it with certainty, the user agent can "recognize" the information. For instance, HTML allows authors to specify a heading with the `h1` element, so a user agent that implements HTML can recognize that content as a heading. If the author creates a heading using a visual effect alone (e.g. just by increasing the font size), then the author has encoded the heading in a manner that does not allow the user agent to recognize it as a heading. Some requirements of UAAG 2.0 depend on content roles, content relationships, timing relationships, and other information supplied by the author. These requirements only apply when the author has encoded that information in a manner that the user agent can recognize. See the section on [conformance](#) for more information about applicability. User agents will rely heavily on information that the author has encoded in a markup language or style sheet language. Behaviors, style, meaning encoded in a [script](#), and markup in an unfamiliar XML namespace can not be recognized by the user agent as easily or at all.

recognized actions: Actions or events that can be unambiguously identified by a user agent. This can include actions or events initiated by users, scripts, add-ons, or other sources. For example, if the keyboard focus is on a web page when the user presses a key, the user agent can recognize the keystroke and can act upon it. If the keyboard focus is on an embedded media player when the user presses a key, the host user agent may or may not be able to detect the keystroke, depending on the embedding architecture. Similarly, when the user activates an `INPUT` element with `type="submit"`, the user agent will recognize this as a form submission action and carry out the proper interchange with the server. However, if a page includes a custom control that looks like a button labeled "Submit*" but whose actions are entirely handled by an author-provided script, the user agent would not be able to recognize the user action as equivalent to a form submission. Actions such as opening of new browser window would always be implemented by the user agent, so the action would be recognized regardless of whether it was initiated by the user clicking a button or by a script calling a browser function.

reflowable content

[Web content](#) that can be arbitrarily wrapped over multiple lines. The primary exceptions to reflowable content are graphics and [video](#).

relative time units

Time intervals for navigating media relative to the current point (e.g. move forward 30 seconds). When interacting with a time-based media presentation, a user can find it beneficial to move forward or backward via a time interval relative to their current position. For example, a user can find a concept unclear in a video lecture and elect to skip back 30 seconds from the current position to review what had been described. Relative time units can be preset by the user agent, configurable by the user, and/or automatically calculated based upon media duration (e.g. jump 5 seconds in a 30-second clip, or 5 minutes in a 60-minute clip). Relative time units are distinct from absolute time values such as the 2 minute mark, the half-way point, or the end.

rendered content

The presentation generated by the user agent based on the author supplied code. This includes:

- content specified by the author
- dynamic content created or altered by scripts

- content inserted by the user agent (e.g. repair text and security warnings on links)
- content inserted by style sheets or markup (e.g. number or letter preceding an ordered list item, the CSS content property)

rendered text: Text content that is rendered in a way that communicates information about the characters themselves, whether visually or as synthesized speech.

repair content, repair text

Content generated by the user agent to correct an error condition. "Repair text" refers to the text portion of repair content. Error conditions that can lead to the generation of repair content include:

- Erroneous or incomplete content (e.g. ill-formed markup, invalid markup, or missing alternative content that is required by format specification);
- Missing resources for handling or rendering content (e.g. the user agent lacks a font family to display some characters, or the user agent does not implement a particular scripting language).

Note: UAAG 2.0 does not require user agents to include repair content in the document object. Repair content inserted in the document object should conform to the Web Content Accessibility Guidelines 2.0 [\[WCAG20\]](#). For more information about repair techniques for web content and software, refer to "Implementing ATAG 2.0" [\[ATAG20-IMPLEMENTING\]](#).

RFC 2119

A publication of the Internet Engineering Task Force (IETF) on Key words for use in Request for Comments (RFC) to Indicate Requirement Levels. The key words are "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL". This information is provided for explanation. UAAG 2.0 does not use these terms as defined in RFC 2119.

script

Instructions to create dynamic web content that are written in a programming (scripting) language. In guidelines referring to the written (natural) language of content, as referenced in Unicode [\[UNICODE\]](#), script can also refer to "a collection of symbols used to represent textual information in one or more writing systems". Information encoded in (programming) scripts can be difficult for a user agent to recognize. For instance, a user agent is not expected to recognize that, when executed, a script will calculate a factorial. The user agent will be able to recognize some information in a script by virtue of implementing the scripting language or a known program library (e.g. the user agent is expected to recognize when a script will open a viewport or retrieve a resource from the web).

selection

A user agent mechanism for identifying a (possibly empty) range of content that will be the implicit source or target for subsequent operations. The selection can be used for a variety of purposes, including for cut-and-paste operations, to designate a specific element in a document for the purposes of a query, and as an indication of point of regard (e.g. the matched results of a search can be automatically selected). The selection should be highlighted in a distinctive manner. On the screen, the selection can be highlighted in a variety of ways, including through colors, fonts, graphics, and magnification. When rendered using synthesized speech, the selection can be highlighted through changes in pitch, speed, or prosody.

source text

Text that the user agent renders upon user request to view the source of specific viewport content (e.g. selected content, frame, page).

style properties

Properties whose values determine the presentation (e.g. font, color, size, location, padding, volume, synthesized speech prosody) of content elements as they are rendered (e.g. onscreen, via loudspeaker, via braille display) by user agents. Style properties can have several origins:

- **user agent default styles:** The default style property values applied in the absence of any author or user styles. Some [web content technologies](#) specify a default rendering; others do not.
- **author styles:** Style property values that are set by the author as part of the content (e.g. in-line styles, [author style sheets](#)).
- **user styles:** Style property values that are set by the user (e.g. via user agent interface settings, [user style sheets](#)).

style sheet

A mechanism for communicating [style property](#) settings for web content, in which the style property settings are separable from other content resources. This separation allows author style sheets to be toggled or substituted, and user style sheets defined to apply to more than one resource. Style sheet [web content technologies](#) include Cascading Style Sheets (CSS) and Extensible Stylesheet Language (XSL).

- **user style sheet:** Style sheets that are not provided by the web content author. The user interface for configuring user style sheets can be targeted at advanced users.
- **author style sheet:** Style sheets specified by the author, resulting in [author styles](#).

synchronize

The act of time-coordinating two or more presentation components (e.g. a [visual track](#) with captions, several tracks in a multimedia presentation). For authors, the requirement to synchronize means to provide the data that will permit sensible time-coordinated rendering by a user agent. For example, web content developers can ensure that the segments of caption text are neither too long nor too short, and that they map to segments of the visual track that are appropriate in length. For user agent developers, the requirement to synchronize means to present the content in a sensible time-coordinated fashion under a wide range of circumstances including technology constraints (e.g. small text-only displays), user limitations (e.g. slow reading speeds, large font sizes, high need for review or repeat functions), and content that is sub-optimal in terms of accessibility.

technology (web content technology)

A mechanism for encoding instructions to be rendered, played or executed by [user agents](#). Web content technologies can include markup languages, data formats, or programming languages that [authors](#) can use alone or in combination to create end-user experiences that range from static web pages to multimedia presentations to dynamic web applications. Some common examples of web content technologies include HTML, CSS, SVG, PNG, PDF, Flash, and JavaScript.

text

A sequence of characters that are [programmatically available](#), where the sequence is expressing something in human language.

- **non-text content:** Any content that is not a sequence of characters that can be programmatically determined or where the sequence is not expressing something in human language Note: This includes ASCII Art (which is a pattern of characters), emoticons, leetspeak (which uses character substitution), and images representing text.
- **blocks of text:** More than one sentence of text [based on [WCAG definition: blocks of text](#)]

text transcript

A type of alternative content that takes the form of text equivalents of audio information (e.g. an audio-only presentation or the [audio track](#) of a movie or other animation). A text transcript provides text for both spoken words and non-spoken sounds such as sound effects. Text transcripts make audio information accessible to people who have hearing disabilities and to people who cannot play the audio. Text transcripts are usually created by hand but can be generated on the fly (e.g. by voice-to-text converters).

top-level viewport

see [viewport](#)

user agent

Any software that retrieves, renders and facilitates end user interaction with web content. UAAG 2.0 identifies the following user agent architectures:

- **platform-based user agent, native user agent:** User agents that run on non-web platforms (operating systems and cross-OS platforms, such as Java) and perform content retrieval, rendering and end-user interaction facilitation themselves (e.g. Firefox, Internet Explorer, Chrome, Opera, Windows Media Player, QuickTime Pro, RealPlayer).
- **embedded user agent, plug-in:** User agents that "plug-in" to other user agents or applications (e.g. media player plug-in for a web browser, web view component). Embedded user agents can establish direct connections with the platform (e.g. communication via platform accessibility services). See [user agent add-on](#).
- **web-based user agent:** User agents that have user interfaces that are implemented using web content technologies and are accessed by users via a user agent. Web-based user agents transform content into web content technologies that the host user agent can render (e.g. web-based ePub reader, web-based video player).

Note 1: Success criteria may also be met by other software. See the applicability notes on [Add-ons \(Extensions and Plug-ins\)](#) and [Relationship with operating system or platform](#).

Note 2: Many web applications retrieve, render and facilitate interaction with very limited data sets (e.g. online ticket booking). In such cases, WCAG 2.0, without UAAG 2.0, can be appropriate for assessing the application's accessibility.

Examples of software that are generally considered user agents under UAAG 2.0:

- Desktop web browsers (e.g. Internet Explorer, Firefox, Chrome, Safari, Opera)
- Mobile web browsers (e.g. Firefox, Chrome, Safari, Android Browser, Opera Mini, Atomic Web, Puffin)
- Browser plug-ins (e.g. QuickTime Plug-in for Firefox, Acrobat Reader Plug-in for Internet Explorer, Shockwave Plug-in for Chrome)
- Web view components (e.g. Webkit Webview component, Web Tools Platform Plug-in for Eclipse, UIWebView for iOS)
- Authoring tools that render the web content being edited (e.g. Word, Dreamweaver, HTML-Kit)

Examples of software that are not considered user agents under UAAG 2.0 (in all cases, WCAG 2.0 still applies if the software is web-based):

- Operating environments or software bundles that include platform-based user agents (e.g. Windows, OS X, KDE, iOS), though the included user agents themselves are covered by UAAG 2.0.
- General-purpose platforms or toolkits that don't use web technologies, even though they can be used by user agents for other purposes (e.g. GNOME, KDE, .NET Framework/CLR).
- Narrow-purpose platform-based or web applications (e.g. online ticket booking applications).
- Authoring tools that only display a source view of the web content being edited (e.g. Notepad, Vim).

user agent add-on (add-in, extension, plug-in)

Software installed into a [user agent](#) that adds one or more additional features that modify the behavior of the user agent. Extensions and plug-ins are types of add-ons. See [embedded user agent](#) and applicability note on [Add-ons \(Extensions and Plug-ins\)](#) for additional information. Two common capabilities for user agent add-ons are the ability to

- modify the [content](#) before the user agent renders it (e.g. to add highlights if certain types of alternative content are present), and
- modify the user agent's own [user interface](#) (e.g. add a headings view).

user interface

For the purposes of UAAG 2.0, the user interface includes both:

- **user agent user interface (UA user interface):** The [controls](#) (e.g. menus, buttons, [prompts](#), native audio/video player controls, and other components for input and output) and mechanisms (e.g. selection and focus) provided by the [user agent](#) that are not created on the basis of the author-supplied content. The UA user interface can include [extensions](#) that become part of the UA user interface (e.g. toolbars, additional menus).
- **content user interface:** The user interface that emerges from the user agent rendering of the author-supplied content. It includes all [rendered content](#) (e.g. [text](#), headings, [enabled elements](#), [disabled elements](#), author-supplied audio/video controls).

Note: There can be a mix of [recognized](#) and unrecognized user interface controls depending on the author-supplied content.

This document distinguishes UA user interface and content user interface only where required for clarity.

user interface control

A component of the [user agent user interface](#) or the [content user interface](#), distinguished where necessary.

video

The technology of moving pictures or images. Video can be made up of animated or photographic images, or both.

view

A user interface function that lets users interact with web content. UAAG 2.0 recognizes a variety of approaches to presenting the content in a view, including:

- **rendered view:** A view where content is presented such that it is rendered, played or executed. There are two sub-types:
 - In conventionally rendered views the content is rendered, played or executed according to the web content technology specification. This is the default view of most user agents.
 - In unconventionally rendered views the content is rendered quite differently than specified in the technology specification (e.g. rendering an audio file as a graphical wavefront).
- **source view:** A view where the web content is presented without being rendered, played or executed. The source view can be plain text (i.e. "View Source") or it can include some other organization (e.g. presenting the markup in a tree).
- **outline view:** A view where only a subset of the rendered content is presented, usually composed of labels or placeholders for important structural elements. The important structural elements will depend on the web content technology, but can include headings, table captions, and content sections.

Note: A view can be visual, audio, or tactile.

viewport

A mechanism for presenting only part of a visual or tactile [view](#) to the user via a screen or tactile display. There can be multiple viewports on to the same underlying view (e.g. when a split-screen is used to present the top and bottom of a document simultaneously) and viewports can be nested (e.g. a scrolling frame located within a larger document). When the viewport is smaller than the view it is presenting, some of the view will not be presented. Mechanisms are typically provided to move the view or the viewport such that all of the view can be brought into the viewport (e.g. scrollbars).

Note: In UAAG 1.0 viewports were defined as having a temporal dimension. In UAAG 2.0, this is not the case. Since audio content is inherently time-based, audio viewports are excluded.

- **top-level viewport:** A viewport that is not contained within another viewport of a platform-based user agent. Web-based user agents are always displayed inside another viewport, and therefore are never top-level viewports. A popular browser implementation is to provide a window that includes some [UA user interface elements](#) (e.g., menus) and a series of tabbed panels, each of which contains additional UA user interface elements (e.g., address bar,

bookmarks, back/forward buttons) and a top-level viewport for rendering a view of the addressed web resource.

viewport dimensions

The onscreen size of a [viewport](#), or the temporal duration of a viewport displaying time-based media. When the dimensions (spatial or temporal) of rendered content exceed the dimensions of the viewport, the user agent provides mechanisms such as scroll bars and advance and rewind controls so that the user can access the rendered content "outside" the viewport (e.g. when the user can only view a portion of a large document through a small graphical viewport, or when audio content has already been played).

visual-only

Content consisting exclusively of one or more [visual tracks](#) presented concurrently or in series (e.g. a silent movie is an example of a visual-only presentation).

visual track

Content rendered through a graphical [viewport](#). Visual objects include graphics, text, and visual portions of movies and other animations. A visual track is a visual object that is intended as a whole or partial presentation. A visual track does not necessarily correspond to a single physical object or software object.

voice browser

A device (hardware and software) that interprets voice markup languages to generate voice output, interpret voice input, and possibly accept and produce other modalities of input and output. Definition from "Introduction and Overview of W3C Speech Interface Framework" [\[VOICEBROWSER\]](#).

web resource

Anything that can be identified by a Uniform Resource Identifier (URI).

Appendix B: How to refer to UAAG 2.0 from other documents

This section is [informative](#).

For the most up-to-date information, see "Referencing and Linking to WAI Guidelines and Technical Documents" at [<http://www.w3.org/WAI/intro/linking.html>](http://www.w3.org/WAI/intro/linking.html).

There are two recommended ways to refer to the "User Agent Accessibility Guidelines 2.0" (and to W3C documents in general):

1. References to a specific version of "User Agent Accessibility Guidelines 2.0." For example, use the "this version" URI to refer to the current document:
<http://www.w3.org/TR/2015/NOTE-UAAG20-20151215/>
2. References to the latest version of "User Agent Accessibility Guidelines 2.0." Use the "latest version" URI to refer to the most recently published document in the series:
<http://www.w3.org/TR/UAAG20/>.

The top of UAAG 2.0 includes the relevant catalog metadata for specific references (including title, publication date, "this version" URI, editors' names, and copyright information).

An XHTML 1.0 paragraph including a reference to this specific document might be written:

```
<p>
<cite><a href="http://www.w3.org/TR/2010/WD-UAAG20-20100617/">
"User Agent Accessibility Guidelines 2.0,"</a></cite>
J. Allan, K. Ford, J. Spellman, eds.,
W3C Recommendation, http://www.w3.org/TR/ATAG20/.
The <a href="http://www.w3.org/TR/ATAG20/">latest version</a> of this
document is available at http://www.w3.org/TR/ATAG20/.</p>
```

For very general references to this document (where stability of content and anchors is not required), it can be appropriate to refer to the latest version of this document. Other sections of this document explain how to build a [conformance claim](#).

Appendix C: References

This section is [informative](#).

For the **latest version** of any W3C specification please consult the list of [W3C Technical Reports](#) at <http://www.w3.org/TR/>. Some documents listed below may have been superseded since the publication of UAAG 2.0.

Note: In UAAG 2.0, bracketed labels such as "[WCAG20]" link to the corresponding entries in this section. These labels are also identified as references through markup.

[ARIA10]

"Accessible Rich Internet Applications (WAI-ARIA) 1.0" J. Craig, M. Cooper, eds., 20 March 2014. This W3C Recommendation is <http://www.w3.org/TR/2014/REC-wai-aria-20140320/>

[ATAG20]

"[Authoring Tool Accessibility Guidelines \(ATAG\) 2.0](#)," J. Richards, J. Spellman, J. Treviranus, eds., 24 September 2015. This W3C Recommendation is <http://www.w3.org/TR/2015/REC-ATAG20-20150924/>

[ATAG20-IMPLEMENTING]

"[Implementing ATAG 2.0](#)" J. Richards, J. Spellman, J. Treviranus, eds., 24 September 2015. This W3C Note is <http://www.w3.org/TR/2015/NOTE-IMPLEMENTING-ATAG20-20150924/>.

[CHARMOD]

"[Character Model for the World Wide Web](#)," M. Dürst and F. Yergeau, eds., 30 April 2002. This W3C Working Draft is <http://www.w3.org/TR/2002/WD-charmod-20020430/>. The [latest version](#) is available at <http://www.w3.org/TR/charmod/>.

[CSS21]

"[Cascading Style Sheets Level 2 Revision 1 \(CSS 2.1\) Specification](#)," B. Bos, T. Celik, I. Hickson, H. Lie, eds., 07 June 2011. This W3C Recommendation is <http://www.w3.org/TR/2011/REC-CSS2-20110607/>.

[DOM2HTML]

"[Document Object Model \(DOM\) Level 2 HTML Specification](#)," J. Stenback, P. Le Hégarret, A. Le Hors, eds., 8 November 2002. This W3C Proposed Recommendation is <http://www.w3.org/TR/2002/PR-DOM-Level-2-HTML-20021108/>. The [latest version](#) is available at <http://www.w3.org/TR/DOM-Level-2-HTML/>.

[HTML4]

"[HTML 4.01 Recommendation](#)," D. Raggett, A. Le Hors, and I. Jacobs, eds., 24 December 1999. This W3C Recommendation is <http://www.w3.org/TR/1999/REC-html401-19991224/>.

[RFC2616]

"[Hypertext Transfer Protocol — HTTP/1.1](#)," J. Gettys, J. Mogul, H. Frystyk, L. Masinter, P. Leach, T. Berners-Lee, June 1999.

[RFC3023]

"[XML Media Types](#)," M. Murata, S. St. Laurent, D. Kohn, January 2001.

[SMIL]

"[Synchronized Multimedia Integration Language \(SMIL\) 1.0 Specification](#)," P. Hoschka, ed., 15 June 1998. This W3C Recommendation is <http://www.w3.org/TR/1998/REC-smil-19980615/>.

[SMIL20]

"[Synchronized Multimedia Integration Language \(SMIL 2.0\) Specification](#)," J. Ayars, et al., eds., 7 August 2001. This W3C Recommendation is

<http://www.w3.org/TR/2001/REC-smil20-20010807/>.

[SVG]

[*"Scalable Vector Graphics \(SVG\) 1.0 Specification,"*](#) J. Ferraiolo, ed., 4 September 2001. This W3C Recommendation is <http://www.w3.org/TR/2001/REC-SVG-20010904/>.

[UAAG10]

[*"User Agent Accessibility Guidelines 1.0,"*](#) I. Jacobs, J. Gunderson, E. Hansen, eds. 17 December 2002. This W3C Recommendation is available at <http://www.w3.org/TR/2002/REC-UAAG10-20021217/>.

[UAAG10-CHECKLIST]

An appendix to UAAG 2.0 lists all of the checkpoints, sorted by priority. The checklist is available in either [tabular form](#) or [list form](#).

[UAAG10-ICONS]

Information about [UAAG 1.0 conformance icons](#) and their usage is available at <http://www.w3.org/WAI/UAAG10-Conformance>.

[UAAG10-SUMMARY]

An appendix to UAAG 1.0 provides a [summary](#) of the goals and structure of User Agent Accessibility Guidelines 1.0.

[UAAG10-TECHS]

[*"Techniques for User Agent Accessibility Guidelines 1.0,"*](#) I. Jacobs, J. Gunderson, E. Hansen, eds. The latest draft of the techniques document is available at <http://www.w3.org/TR/UAAG10-TECHS/>.

[UNICODE]

The Unicode Consortium. [*The Unicode Standard, Version 8.0.0*](#), (Mountain View, CA: The Unicode Consortium, 2015. ISBN 978-1-936213-10-8) link: <http://www.unicode.org/versions/Unicode8.0.0/>

[VOICEBROWSER]

[*"Introduction and Overview of W3C Speech Interface Framework,"*](#) J. Larson, 4 December 2000. This W3C Working Draft is <http://www.w3.org/TR/2000/WD-voice-intro-20001204/>. The [latest version](#) is available at <http://www.w3.org/TR/voice-intro/>. UAAG 2.0 includes references to additional W3C specifications about voice browser technology.

[W3CPROCESS]

[*"World Wide Web Consortium Process Document,"*](#) I. Jacobs ed. The 19 July 2001 version of the Process Document is <http://www.w3.org/Consortium/Process-20010719/>. The [latest version](#) is available at <http://www.w3.org/Consortium/Process/>.

[WCAG20]

[*"Web Content Accessibility Guidelines \(WCAG\) 2.0"*](#) B. Caldwell, M. Cooper, L. Guarino Reid, G. Vanderheiden, eds., 8 December 2008. This W3C Recommendation is <http://www.w3.org/TR/2008/REC-WCAG20-20081211/>. The [latest version](#) is available at <http://www.w3.org/TR/WCAG20/>. Additional format-specific techniques documents are available from this Recommendation.

[WCAG20-TECHS]

[*"Techniques for Web Content Accessibility Guidelines 2.0,"*](#) B. Caldwell, M. Cooper, L. Guarino Reid, G. Vanderheiden, eds., 8 December 2008. This W3C Note is <http://www.w3.org/TR/2010/NOTE-WCAG20-TECHS-20101014/>. The [latest version](#) is available at <http://www.w3.org/TR/WCAG20-TECHS/>. Additional format-specific techniques documents are available from this Note.

[WCAG-EM]

[*"Website Accessibility Conformance Evaluation Methodology \(WCAG-EM\) 1.0"*](#) E. Velleman, S. Abou-Zahra, eds., 26 February 2013. This is an informative draft of a Working Group Note. The [latest version](#) is available at <http://www.w3.org/TR/WCAG-EM/>

[WCAG2ICT]

[*Guidance on Applying WCAG 2.0 to Non-Web Information and Communications Technologies \(WCAG2ICT\)*](#) M. Cooper, P. Korn, A. Snow-Weaver, G. Vanderheiden,

eds., 5 September 2013. This document is available in an [expandable / collapsible alternate version](#) in which the “Intent” sections copied from Understanding WCAG 2.0 are hidden and individually expandable, for easier reading.

[WEBCHAR]

["Web Characterization Terminology and Definitions Sheet,"](#) B. Lavoie, H. F. Nielsen, eds., 24 May 1999. This is a W3C Working Draft that defines some terms to establish a common understanding about key Web concepts. This W3C Working Draft is <http://www.w3.org/1999/05/WCA-terms/01>.

[XAG10]

["XML Accessibility Guidelines 1.0,"](#) D. Dardailler, S. Palmer, C. McCathieNevile, eds., 3 October 2001. This W3C Working Draft is <http://www.w3.org/TR/2002/WD-xag-20021003>. The [latest version](#) is available at <http://www.w3.org/TR/xag>.

[XML]

["Extensible Markup Language \(XML\) 1.0 \(Second Edition\),"](#) T. Bray, J. Paoli, C.M. Sperberg-McQueen, eds., 6 October 2000. This W3C Recommendation is <http://www.w3.org/TR/2000/REC-xml-20001006>.

[XHTML10]

["XHTML\[tm\] 1.0: The Extensible HyperText Markup Language,"](#) S. Pemberton, et al., 26 January 2000. This W3C Recommendation is <http://www.w3.org/TR/2000/REC-xhtml1-20000126/>.

[XMLDSIG]

["XML-Signature Syntax and Processing,"](#) D. Eastlake, J. Reagle, D. Solo, eds., 12 February 2002. This W3C Recommendation is <http://www.w3.org/TR/2002/REC-xmlsig-core-20020212/>.

[XMLENC]

["XML Encryption Syntax and Processing,"](#) D. Eastlake, J. Reagle, eds., 10 December 2002. This W3C Recommendation is <http://www.w3.org/TR/2002/REC-xmlenc-core-20021210/>.

Appendix D: Acknowledgments

Participants active in the UAWG prior to publication:

- Jim Allan (Chair, Texas School for the Blind and Visually Impaired)
- Eric Hansen (Educational Testing Service)
- Greg Lowney (Invited Expert)
- Kimberly Patch (Invited Expert)
- Jan Richards (Inclusive Design Institute, OCAD University)
- Jeanne Spellman (W3C Staff Contact)

Previous Editors:

Kelly Ford, Microsoft
Jan Richards, Inclusive Design Institute, OCAD University

Additional Contributors of Mobile Examples

- Kathy Walhbin
- Mark Sadecki

Other previously active UAWG participants and other contributors to UAAG 2.0:

- Judy Brewer (W3C)
- Alan Cantor (Invited Expert)
- Wayne Dick (Invited Expert)

- Bim Egan (Royal National Institute of Blind People)
- Kelly Ford (Microsoft)
- Mark Hakkinen (Educational Testing Service)
- Simon Harper (University of Manchester)
- Sean Hayes (Microsoft)
- Dean Hudson (Apple)
- Patrick Lauke (Opera Software)
- Cathy Laws (IBM)
- Peter Parente (IBM)
- David Poehlman (Invited Expert)
- Simon Pieters (Opera Software)
- Henny Swan (Opera Software)
- Gregory Rosmaita (Invited Expert)
- David Tseng (Apple)

UAAG 2.0 would not have been possible without the work of [those who contributed to UAAG 1.0](#).

This publication has been funded in part with Federal funds from the U.S. Department of Health and Human Services, National Institute on Disability Independent Living and Rehabilitation Research (NIDILRR) under contract HHSP23301500054, and previously by the Department of Education's NIDILRR contracts ED05CO0039 and ED-OSE-10-C-006. The content of this publication does not necessarily reflect the views or official policies of the U.S. Department of Education or U.S. Department of Health and Human Services, nor does mention of trade names, commercial products, or organizations imply endorsement by the U.S. Government.
